Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی دنیا کے Claude-based وقت درست حل نافذ کرتا ہے یہ امتحان trade-off فیصلہ کر سکتا ہے۔ Claude Code، Model Context Protocol (MCP) یعنی — یہ کہ بنیادی علم کا جائزہ لیتا ہے Claude Agent SDK، Claude API، اور کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں Claude وہ بنیادی ٹیکنالوجیز جن سے

امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے agentic systems بنانا، multi-agent research pipelines ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity tools بنانا، اور غیر ساختہ دستاویزات سے structured data نکالنا

ہدف امیدوار

مثالی امیدوار ایک **solution architect** جو ہے Claude اور پروڈکشن ایپلیکیشنز ڈیزائن کے ساتھ ship آپ کرتا ہے جو کہ پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات کے ساتھ کام، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 درست 1 میں سے)
اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	8 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent بنا رہے ہیں جو Claude Agent SDK استعمال کرتا ہوا returns، billing disputes، اور account issues کو سنبھالتا ہے۔ agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) استعمال کرتا ہے۔ ہدف 80%+ first-contact resolution مناسب escalation کے ساتھ۔

Scenario 2: Claude Code کے ساتھ Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں تیز کرنے کے لیے: code generation، refactoring، debugging، documentation۔ آپ کو اس کے ساتھ CLAUDE.md configuration اور custom slash commands integrate کرنے کے لیے سمجھنا ہے کہ planning mode کب استعمال کرنا ہے، اور یہ کہ کب کرنا ہے۔

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks سونپتا ہے: web research، document analysis، synthesis، اور report generation۔ سسٹم کو citations کے ساتھ مکمل reports تیار کرنے چاہئیں۔

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، اور MCP servers اور Built-in tools (Read، Write، Bash، Grep، Glob) استعمال کرنے میں مدد دیتا ہے automate کیے جاتے ہیں۔

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback حاصل ہو۔ Prompts کے ایسے ڈیزائن ہونے چاہئیں کہ false positives کم سے کم ہوں۔

Scenario 6: Structured Data Extraction

□□ معلومات نکالتا □ سے دستاویزات □ سسٹم غیر ساخت□، output کو JSON schemas □ ذریعہ □ کے validate ،□□ کرتا اور high accuracy □ اسے □□□ برقرار رکھتا edge cases □□ یہ□ چاہ□ کو درست طور پر سنبھالنا

Scenario 7: Conversational AI Architecture Patterns

آپ multi-turn conversational systems میں جن □یں □ ڈیزائن کرتے context window management، متعدد turns میں instructions □نا برقرار ر□ کا، memory strategies، محفوظ execution □ لیے □ کے tool design، اور مبہ□م یا متضاد user inputs □□□ کو سنبھالنا شامل

Scenario 8: Agentic AI Tools *(مواد موجود نہیں — ہماری مدد کریں!)*

اس scenario □ والے candidates کی اطلاع امتحان دینے □ ابھی تک اس □ لیکن یہ □□ دی □ نہ guide میں شامل نہ□یں کریں share میں GitHub Issues کے □ سوالات دیکھ□ □یں، تو ان□یں □□□ scenario اگر آپ نہ □ اصل امتحان میں اس □م مکمل □ تاکہ coverage کر تیاری کی امتحان جو گا □ مدد کر□ شخص کی اس ر□ تعاون کا آپ □ شامل کر سکیں ر□ا □□□

سرکاری دستاویزات

وسیلہ□	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills

Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

---

□صہ I: نظریاتی بنیادیں

یہ حصہ وہ تمام نظریہ احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے مواد کو ٹیکنالوجیز اور concepts امتحانی کے مطابق ترتیب دیا گیا ہے، نہ کہ domains کے مطابق — اس سے آپ کو موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API ایک request–response model ہے جو Claude Messages API کی پیروی کرتا ہے۔ ہر request میں یہ شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens کی تعداد
- `system` — system prompt (model کے رویے کی تعریف کرتا ہے)
- `messages` — conversation history (**تسلسل برقرار رکھنے کے لیے آپ کو مکمل history بھیجنی ہوتی** ہے)
- `tools` — دستیاب tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array تین roles استعمال کرتا ہے:

- `user` — user messages
- `assistant` — model responses (history میں شامل کیے جاتے وقت بھیجتے ہیں)
- `tool` — tool call results (role واضح طور پر set ہے؛ `tool_result` content block کی صورت میں ظاہر ہوتا ہے)

انتہائی اہم: ہر API request میں آپ کو **مکمل conversation history** بھیجنی چاہیے Model requests کے درمیان state ہر — محفوظ نہیں رکھتا call آزاد ہوتا

1.3 Response میں `stop_reason` Field

Claude API response میں `stop_reason` کیوں روکی generation نے model شامل ہوتا ہے، جو بتاتا ہے کہ:

Value	Description	Action
`"end_turn"`	Model نے response مکمل کر لیا	نتیجے user کو دکھائیں
`"tool_use"`	Model tool call کرنا چاہتا ہے	Tool execute کریں اور result واپس دیں
`"max_tokens"`	Token limit ختم ہو گئی	Response truncated ہے ممکن ہے؛ limit بڑھانی پڑے
`"stop_sequence"`	Stop sequence مل گئی	اپنی application logic کے مطابق handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` ہی — یہی سب سے اہم ہیں جو agent loop کو کنٹرول کرتے ہیں۔

1.4 System Prompt

System prompt ایک خاص instruction جو ہے context اور behavioral rules ہے یہ متعین کرتا ہے:

- `messages` array الگ ہے یہ ہوتا؛ نہیں کا حصہ `system` field ہے میں دیا جاتا ہے
- user messages ہے پر فوقیت رکھتا ہے
- ایک بار load اور پوری ہے ہوتا conversation ہے میں لاگو رہتا ہے
- role، constraints، اور output format ہے ہوتا استعمال ہے لیے کے لیے کی تعریف کے

امتحان کے لیے اہم: system prompt کی wording غیر ارادی tool associations طور کے مثال کے پیدا کر سکتی ہے ہیں۔ جیسی ہدایت "customer پر، کی تصدیق کریں" کو model `get_customer` کو ضرورت سے زیادہ استعمال کرنے پر مجبور کر سکتی ہے، حتیٰ کہ جب یہ ضروری نہ ہو۔

1.5 Context Window

Context window text (tokens) ہے جسے ہے کی کل مقدار ہے ایک وقت میں model کی process اس میں ہے کر سکتا ہے ہیں۔ شامل ہے:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

اہم context-window مسائل:

1. **Lost-in-the-middle effect:** models لمبے input سے اعتماد کو زیادہ کے آغاز اور اختتام پر موجود معلومات کو زیادہ کر سکتے ہیں۔ حل: اہم معلومات شروع یا آخر کے قریب رکھیں، مگر درمیان کی تفصیلات process کرتے ہیں
2. **Tool results کا جمع ہونا:** ہر tool call context میں output اگر 40+ fields واپس کرے شامل کرتا ہے کا بڑا حصہ ضائع ہو جاتا ہے context مگر صرف 5 اہم ہوں، تو
3. **Progressive summarization:** history compress وقت کرتے numeric values، percentages، اور dates اکثر گم ہو کر غیر واضح ہو جاتے ہیں ("تقریباً"، "کچھ"، "چند")

باب 2: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` □□ کیا

`tool_use` ایک mechanism جو □□ Claude کو external functions call □□□ کرنے کی اجازت دیتا □□ برا□ Model کرتا result □□ اور execute اسے code □□ بناتا □□؛ آپ کا structured tool call request نہیں چلاتا — وہ code راست واپس کرتا □□□

2.2 Tool Definition

ہر tool ایک JSON schema □ذریعے □کے define □□ کیا جاتا:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description □□ انتہائی اہم پہلو ہے:

1. **Description tool selection کا بنیادی mechanism □□□** LLM tools کو ان کی descriptions کی بنیاد پر □□□ منتخب کرتا Minimal descriptions (“Customer information retrieve □□ کرتا”) اُن مواقع پر غلطیوں کا باعث بنتی □یں جب tools □سے ایک دوسرے □سے overlap کرتے □وں

2. **Description میں شامل کریں:**
   - Tool □□ کیا کرتا □□ اور کیا واپس کرتا
   - Input formats اور example values
   - Edge cases اور constraints
   - □ہ tool similar alternatives □و کے مقابلے میں کب استعمال □□

3. **یا □جیسے ایک جیسے overlapping descriptions** اگر □آپ کے سے `analyze_content` اور `analyze_document` کی descriptions تو، □وں جیسی ایک تقریباً model □گا □د کر ملط خلط □یں□ان

4. **Built-in tools □مقابلے MCP tools:** agents similar functionality □والے built-in tools (Read، Grep) کو MCP tools □لیے کہ □سے بچنے اس □یں□ سکتے □د ترجیح پر MCP tool descriptions بنائیں مضبوط — concrete advantages، unique data، □ہ یا context جو نمایاں کریں built-in tools □□سکتے کر □یں نہ فراہم

2.3 `tool_choice` Parameter

`tool_choice` □کہ □□ کنٹرول کرتا □ہ model tools □□ منتخب کرتا □کیسے:

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model کہ کرنا tool call خود طے کرتا ہے یا text میں جواب دینا ہے	default زیادہ تر حالات میں
`{ "type": "any" }`	Model کو لازماً کوئی tool call کرنا ہوگی	جب آپ کو guaranteed structured output چاہیے
`{ "type": "tool", "name": "extract_metadata" }`	Model کو لازماً ایک مخصوص tool call کرنی ہوگی	جب آپ کو forced first step / execution order چاہیے

ہم منظرنامہ:

- `tool_choice: "any"` + multiple extraction tools → model بہترین tool منتخب کرتا ہے، مگر پھر بھی structured output ملتا ہے
- Forced selection → جب آپ کو کسی خاص پہلی action کی ضمانت چاہیے (مثلاً enrichment سے پہلے `extract_metadata`)

2.4 Structured Output کے لیے JSON Schemas

JSON schemas کے ساتھ `tool_use` استعمال کرنا Claude سے structured output حاصل کرنے کا **سب سے قابلِ اعتماد** طریقہ ہے:

- Syntax درست ہے اعتبار کہ JSON کی ضمانت دیتا ہے (braces ہوتے ہیں غائب، trailing commas نہیں ہوتے)
- مطلوبہ structure نافذ کرتا ہے (required fields ہیں ہوتے موجود)
- Semantic correctness کی ضمانت نہیں دیتا (values ہیں سکتی ہو غلط پھر بھی)

Schema design — بنیادی اصول:

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

Schema design rules:

1. **Required بمقابلہ optional:** صرف اُن fields کو required بنائیں جن کی معلومات ہمیشہ دستیاب ہو۔ غائب values ہونے پر data کو Required fields model مجبور کر سکتی ہیں گھڑنے پر
2. **Nullable fields:** ایسی معلومات کے لیے `"type": ["string", "null"]` استعمال کریں جو ممکنہ طور پر بنائے value واپس کر سکتا ہے بجائے اس کے کہ وہ خود ساختہ `null` موجود نہ ہوں Model
3. **Enums with `"other"`:** اپنی predefined categories کے لیے سب باہر کی معلومات ضائع ہونے سے بچانے کے لیے شامل کریں ایک detail string + `"other"`
4. **Enum `"unclear"`:** جب اُن صورتوں کے لیے model category ایماندار نہ ہو — جو کہ بار میں پُراعتماد غلط category سے بہتر ہے `"unclear"`

2.5 Syntax بمقابلہ Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema کے ساتھ `tool_use` (ختم کر دیتا ہے)
Semantic	Totals match نہیں کرتے، value غلط field میں، hallucination	Validation checks، feedback ساتھ کے retry، self-correction

3 باب: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop □□ کیا

Agentic loop خودکار task execution کا بنیادی pattern □□□ Model □و — صرف جواب نہ□یں دیتا actions کی ایک sequence □□ انجام دیتا:

```
1. Claude کو tools کے ساتھ request بھیجیں
2. Response وصول کریں
3. stop_reason چیک کریں:
   - "tool_use" -> tool execute کریں، result history میں append کریں، step 1 پر واپس جائیں
   - "end_turn" -> task مکمل، result user کو دکھائیں
4. مکمل ہونے تک repeat کریں
```

□ی **model-driven approach** □□: Claude □اپنے context □پچھلے اور tool results کی بنیاد پر اگلا tool خود منتخب □وتی □□□ fixed پ□لے سے □سی action sequence مختلف □□ جگہ□ں hard-coded decision trees کرتا □□□ □ی

Anti-patterns (بچیں سے □ان):

- Completion detect □لیے کرنے کے □ assistant text parse کرنا ("Task completed")
- Primary stop condition طور پر □ arbitrary iteration limit استعمال کرنا (مثلاً `max_iterations=5`)
- □ assistant □ی textual content □اسے، نہ□یں، دیا □□ completion signal سمجھنا چیک کرنا □

□درست طریقہ: completion کا واحد قابلِ اعتماد signal `stop_reason == "end_turn"` □□□

3.2 `AgentDefinition` Configuration

`AgentDefinition` Claude Agent SDK میں agent configuration object □□:

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # Coordinator میں لازمی:
    # allowed_tools=["Task", "get_customer", ...]
)
```

م□ل parameters:

- `name` / `description` — agent کی شناخت اور وضاحت
- `system_prompt` — □دایات والا system prompt
- `allowed_tools` — □اجازت یافتہ tools کی فہ□رست (least privilege اصول)

3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture عموماً hub-and-spoke topology □□ میں بنائی جاتی ہے:

```
        Coordinator
       /     |      \
  Subagent1  Subagent2  Subagent3
   (search)   (analysis)   (synthesis)
```

Coordinator کی ذمہ□ داریاں:

- Task کو subtasks میں تقسیم کرنا
- □س□ طرح کرنا کہ□ کون سے subagents □یں درکار (dynamic selection)
- کام subagents کو سونپنا
- نتائج جمع اور validate کرنا
- Errors اور retries handle کرنا
- نتائج user تک پ□نچانا

اصول□: subagents کا context □□□ ہوتا □الگ

- Subagents خود بخود coordinator کی conversation history inherit □نہیں کرتے
- تمام ضروری context subagent prompt □□ دینا ہوتا **واضح طور پر** میں
- Subagents calls درمیان □کسی memory share □نہیں کرتے
- تمام communication coordinator □□ ذریعہ□ گزرتی □کہ (observability اور error control لیے □کہ)

3.4 Subagents بنانے کے لیے `Task` Tool

Subagents `Task` tool ذریعہ□ کے spawn □یں جاتے □کیے:

```
# Coordinator کے allowedTools میں "Task" ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing □□ لازمی:

```
# Bad: subagent کو پاس کیا context ہی نہیں
Task: "Analyze the document"

# Good: مکمل context prompt میں
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

Parallel spawning: coordinator ایک response میں متعدد `Task`s call □□ کر سکتا ہے — subagents parallel میں □چلتے □یں:

```
# Coordinator ایک کہ response ہے شامل ہے میں:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

3.5 Agent SDK میں Hooks

Hooks agent lifecycle مخصوص کے points پر interception اور transformation کی اجازت دیتے ہیں۔

PostToolUse tool result کو model تک پہنچانے سے پہلے intercept کرتا ہے:

```
# مختلف :مثال MCP tools کہ تاریخ سے formats normalize کریں
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy والے کی خلاف ورزی کرنے actions کو block کرتا ہے:

```
# اوپر سے 500$ :مثال refunds block کریں
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

Attribute	Hooks	Prompt instructions
Guarantee	**Deterministic** (100%)	**Probabilistic** (>90%, 100% نہیں)
کب استعمال کریں	Critical business rules، financial operations، compliance	General preferences، recommendations، formatting
مثال	Refunds > $500 block کریں	“Try to solve before escalating”

استعمال کریں hooks، نہیں prompts — کہ مالی، قانونی، یا حفاظتی نتائج ہوں failure جب **قاعدہ:**

باب 4: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP □□ کیا

Model Context Protocol (MCP) ایک open protocol جو □□ external systems کو Claude □□□ جوڑتا □ سے MCP تین بنیادی resource types define □□ کرتا:

1. **Tools** — functions جن□یں agent actions □ لیے □ کے انجام دینے call □□ کر سکتا (CRUD operations، API calls، command execution)
2. **Resources** — context □ لیے □ کے data □ جسے agent □□ پڑھ سکتا (documentation، database schemas، content catalogs)
3. **Prompts** — عام tasks □ لیے □ کے predefined prompt templates

4.2 MCP Servers

MCP server ایک process جو □□ MCP protocol implement اور □□ کرتا tools/resources جب □□□ فرا□م کرتا آپ MCP server □ سے connect □یں □ کرتا:

- تمام tools خود بخود discover □یں □ جاتا □و
- تمام connected servers □ کے tools □یں □ وتا □ ایک ساتھ دستیاب
- Tool descriptions □ کے □یں □ کرتی ط□ model گا استعمال کر□ کیسے ان□یں

4.3 MCP Servers کو Configure کرنا

Project configuration (`.mcp.json`) — team usage □ لیے □ کے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

اہ□م نکات:

- `.mcp.json` project root اور □□ وتا □ میں محفوظ version control □□ وتا □ میں
- Environment variables (`${GITHUB_TOKEN}`) secrets □یں □ وتا □ استعمال □ لیے □ کے — tokens خود commit ن□یں کیے جاتے □

- یہ project تمام کے contributors کو دستیاب ہونا چاہیے

User configuration (`~/.claude.json`) — personal/experimental servers کے لیے:

- User کے home directory میں محفوظ ہوتا ہے
- Version control کے ذریعے share نہیں ہوتا
- Personal experiments اور testing موزوں کے لیے

Servers کا انتخاب:

- Standard integrations (Jira، GitHub، Slack) موجود سے پہلے پہلے کے لیے community MCP servers کو ترجیح دیں
- اپنی custom servers صرف unique، team-specific workflows بنائیں کے لیے

4.4 MCP میں `isError` Flag

جب MCP tool میں error تو آتا ہے response میں `isError: true` یہ ہوتا ہے استعمال agent کو بتاتا ہے کہ call fail ہوئی ہے۔

Structured error (اچھا):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the
orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

Generic error (anti-pattern):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

Generic error agent کیا — نہیں دیتا کوئی معلومات کے لیے فیصلہ سازی کہ کو retry کریں، query یا بدلیں، escalate کریں؟

4.5 MCP Resources

Resources ایسا data جو ہیں agent context کے لیے کہ حاصل کرنے actions کے بغیر طلب کر سکتا ہے:

- Content catalogs (مثلاً تمام project tasks کی فہرست، hierarchical navigation)

- Database schemas (data structure کو سمجھنے کے لیے)
- Documentation (API references، internal guides)
- Issue/task summaries

Resource کا فائدہ: agent کو یہ سمجھنے کے لیے exploratory tool calls کی ضرورت نہیں پڑتی کہ data موجود ہے یا نہیں۔ فوری “map” فراہم کرتا ہے کہ Resource کیا ہے

باب 5: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

Claude Code کی تین سطحی hierarchy استعمال کرتا ہے جو instruction file(s) ہیں اور CLAUDE.md ہے:

```
1. User-level: ~/.claude/CLAUDE.md
   - صرف اسی user پر لاگو
   - VCS کے ذریعے share نہیں ہوتا
   - ذاتی preferences اور working style

2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
   - تمام project contributors پر لاگو
   - VCS کے ذریعے manage ہوتا ہے
   - Coding standards، testing standards، architectural decisions

3. Directory-level: subdirectories میں CLAUDE.md
   - جب اسی directory کی files پر تو لاگو ہو کام کے ساتھ
   - codebase کے مخصوص حصے کے conventions
```

عام غلطی: ایک نئے team member کو project instructions دیں ملتیں کیونکہ `.claude/CLAUDE.md` (project-level) کے بجائے `~/.claude/CLAUDE.md` (user-level) میں رکھی گئی تھیں

5.2 `@path` Syntax (File Imports)

CLAUDE.md بیرونی files کو `@path` کے ذریعے refer کر سکتا ہے، جس سے configuration modular ہو جاتی ہے:

```
# Project CLAUDE.md

Coding standards @./standards/coding-style.md میں بیان کیے گئے ہیں
Test requirements @./standards/testing-requirements.md میں ہیں
Project overview @README.md اور dependencies @package.json میں ہیں
```

`@path` قواعد کا:

- `@` فوراً بعد کا file path آئے (کوئی space نہیں)
- Relative اور absolute دونوں paths supported ہیں
- Relative paths اُس file کے مطابق resolve کی جاتی ہیں جس میں import ہوتا ہے
- Maximum import nesting depth 5 ہے

اس سے duplication کم ہوتی ہے اور ہر package میں صرف متعلقہ standards شامل ہوتے ہیں

5.3 `.claude/rules/` Directory

`.claude/rules/` monolithic CLAUDE.md کے متبادل ہے، جو rules کو topic کے حساب سے organize کرنے کا بہتر استعمال ہوتا ہے:

```
.claude/rules/
  testing.md          -- testing conventions
  api-conventions.md  -- API conventions
  deployment.md       -- deployment rules
  react-patterns.md   -- React patterns
```

YAML frontmatter کے ساتھ `paths` conditional loading:

```
---
paths: ["src/api/**/*"]
---

API files کے لیے async/await کے ساتھ explicit error handling استعمال کریں
ہر endpoint standard response wrapper واپس کرے
```

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---

Tests میں describe/it blocks استعمال ہونے چاہئیں
Data factories استعمال کریں، hardcoding نہیں
Database mock نہ کریں — test database استعمال کریں
```

یہ کیسے کام کرتا ہے:

- Rule صرف اس وقت load جب ہوتا ہے Claude Code ایسا file edit جو کرے `paths` pattern سے match کرتا ہو
- اس سے context اور tokens غیر متعلقہ — میں بچتے ہیں rules load ہوتے ہیں
- Glob patterns آپ کو file type کے لحاظ سے conventions apply چاہے، دیتے ہیں کرنے files codebase میں کہیں بھی ہوں (tests مفید بہت لیے کے)

`paths` والے `.claude/rules/` بمقابلہ directory-level CLAUDE.md کب استعمال کریں:

- `.claude/rules/` with `paths` — جب conventions کئی directories میں بکھری files (tests، migrations) پر لاگو ہوں
- Directory-level CLAUDE.md — جب conventions کسی خاص directory سے وابستہ ہوں اور کہیں اور ضروری نہ ہوں

5.4 Custom Slash Commands اور Skills

> **نوٹ:** موجود Claude Code version میں custom commands (`.claude/commands/`) کو skills (`.claude/skills/`) کے ساتھ unify کر دیا گیا ہے۔ دونوں `/name` commands بناتے ہیں اور اب بھی supported ہے format کا ذکر کرتی ہے — وہ `.claude/commands/` امتحانی گائیڈ ہیں۔

Slash commands reusable prompt templates ہیں جو `/name` کے ذریعے invoke ہوتے ہیں:

`.claude/commands/` format (legacy، supported):

```
.claude/commands/
  review.md        -- /review -- standard code review
  test-gen.md      -- /test-gen -- test generation
```

`.claude/skills/` format (current):

```
.claude/skills/
  review/SKILL.md  -- /review -- frontmatter configuration کے ساتھ
  test-gen/SKILL.md
```

Project commands (`.claude/commands/` یا `.claude/skills/`):

- VCS اور ہیں ہوتے دستیاب میں repo clone ہیں کرنے والے سب کے لیے محفوظ ہوتے
- ٹیم میں consistent workflows ہیں یقینی بناتے

User commands (`~/.claude/commands/` یا `~/.claude/skills/`):

- ذاتی commands جو VCS کے ذریعے share ہوتے نہیں
- انفرادی workflows کے لیے

5.5 Skills — `.claude/skills/`

Skills advanced commands ہیں جو SKILL.md frontmatter کے ذریعے configure ہوتے ہیں:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---

دی گئی directory میں code structure کا تجزیہ کریں۔
Dependencies اور architectural patterns پر ایک report تیار کریں۔
```

Frontmatter parameters:

Parameter	Description
`context: fork`	Skill کو isolated subagent میں چلاتا ہے۔ Verbose output main session کو pollute کرتا نہیں
`allowed-tools`	محدود کرتا ہے کہ کون سے tools دستیاب ہوں گے (security — مثلاً skill files delete کر نہیں سکتی اگر اجازت نہ ہو)
`argument-hint`	جب skill parameters کے بغیر invoke ہو تو argument کا اشارہ مانگتا ہے

Skill بمقابلہ CLAUDE.md کب استعمال کریں:

- **Skill** — مخصوص task کے لیے on-demand invocation (review، analysis، generation)
- **CLAUDE.md** — ہر session لوڈ ہونے والے general standards اور conventions

Personal skills (`~/.claude/skills/`):

- اپنی personal variants بنائیں تاکہ مختلف ناموں سے teammates کو متاثر نہ ہوں

5.6 Planning Mode بمقابلہ Direct Execution

Planning mode:

- Model صرف investigate اور plan کرتا ہے؛ changes کرتا نہیں
- Codebase explore کے لیے Read، Grep، Glob کا استعمال کرتا ہے
- ایک implementation plan جسے user approve کرتا ہے تیار کرتا ہے
- بغیر side effects کے safe exploration

Planning mode کب استعمال کریں:

- بڑے changes (درجنوں files)
- Multiple plausible approaches (microservices: boundaries کیسے define ہوں؟)
- Architectural decisions (کون سا framework؟ کیا structure؟)
- Unfamiliar codebase (change سے پہلے سمجھنا ضروری ہے کہ کرنا ہے)
- Library migrations 45 جو +files کو متاثر کریں

Direct execution کب استعمال کریں:

- Single-file fixes واضح stack trace ساتھ □کا
- ایک validation check شامل کرنا
- اچھی طرح سمجھ□ گئ□، غیر مبہ□م changes

Combined approach:

1. Investigation اور design □لی□ کا planning mode
2. User plan approve □کر
3. Approved plan implement □لی□ کا □کرنا direct execution

Explore subagent — codebase explore □لی□ کا □کرنا specialized subagent:

- Verbose output کو main context □سے isolate □□ کرتا
- صرف summary □□ واپس کرتا
- Multi-phase tasks میں context-window exhaustion □□ روکتا

5.7 `/compact` Command

`/compact` context compress □لی□ کا □کرنا built-in command □□:

- Pچھلی history کو summarize □کر کر context window □□ میں جگ□ خالی کرتا
- Long investigation sessions جب □□ □ہوتا میں استعمال verbose tool output □سے context □بھر جائ□
- Risk: exact numeric values، dates، اور specific details summarization □یں □ہو سکت□ میں گم

5.8 `/memory` Command

`/memory` sessions درمیان □کا memory manage □لی□ کا □کرنا built-in command □□:

- `CLAUDE.md` file editing □تاکا، □□ کھولتا □لی□ کا notes، preferences، اور context محفوظ کی□ جا سکیں
- Information sessions اور □□ رہ□تی پار برقرار □کا startup پر خود بخود load □□ ہوتی□
- Project conventions، user preferences، frequently used commands، اور current work context محفوظ کرنا □کا □لی□ مفید
- □ر session میں وہ□ی instructions کا متبادل دوبار□ سمجھانا

5.9 Claude Code CLI for CI/CD

`-p` (یا `--print`) flag:

```
claude -p "Analyze this pull request for security issues"
```

- Non-interactive mode: prompt process □□ کرتا، stdout پر print اور، □□ کرتا exit □□ ہو جاتا□
- User input کا انتظار نہ□یں کرتا
- CI/CD pipelines میں Claude □چلانا کا واحد درست طریق□

CI □لی□ کا structured output:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — output کو JSON میں دیتا ہے
- `--json-schema` — output کو schema مطابق کی validate کرتا ہے
- Result کو automatically inline PR comments پوسٹ کرنے کے لیے parse کیا جا سکتا ہے

Session context isolation: وہی Claude session جس نے code generate کیا ہو، review اکثر میں کم مؤثر ہوتی ہے (model اپنی reasoning context فیصلوں کو اور اپنے ساتھ رکھتا ہے challenge کرنے کا امکان کم ہوتا ہے)۔ Review کے لیے independent instance استعمال کریں

Duplicate comments سے بچاؤ: نئے commits کے بعد دوبارہ review کرتے وقت، پچھلے review results context کرنے کی ہدایت دیں کو صرف نئے یا unresolved issues report میں شامل کریں اور Claude

5.10 Session Management اور `fork_session`

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- saved context پچھلی کے ساتھ conversation جاری رکھتا ہے
- Long investigations جو متعدد sessions مفید ہے کے لیے ان کو پر پھیلی ہوں
- Risk: اگر پچھلی session بعد کے files تو بدل چکی ہوں tool results stale میں ہو سکتے ہیں

`fork_session` shared context سے independent branch بناتا ہے:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit میں کرتے ہیں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- Approaches compare یا کرنے strategies test مفید ہے کے لیے

Resume نئی بجائے کے session کب شروع کریں:

- Tool results stale ہوں (بدل چکی ہوں files)
- کافی وقت گزر چکا ہو اور context degrade ہو گیا ہو
- پرانے tool data کے ساتھ resume کرنے کے بجائے یہ بہتر ہے کہ "ہم نے جو پایا اس کا مختصر خلاصہ یہ ہے: کیا جائے restart ..." سے

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting جس میں □□ □طریقہ □و expected behavior □لیے □کہ □دکھانہ prompt 4–2 میں input/output examples □یں□ جاتے □□ شامل کیے

Few-shot text descriptions □□ مؤثر کیوں □زیاد□ سے□:

- □م□ instruction □زیاد□ “ جیسے□ precise □□ سمجھا جا سکتا سے□ طریقوں کئی کو □وں”
- Example پر طور □م□غیر expected format اور decision logic □□ دکھاتا
- Model pattern □نئ□ کو cases پر generalize □□ کرتا (صرف examples □یں□نہ□راتا)

Few-shot examples کی اقسام اور استعمال:

1. **Ambiguous scenarios □لیے □کہ examples:**

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: “broken” damaged item کو آپ ؛؛ سکتا ہو بھی order details ؍میں درکار

Request: "Get me a manager"
Action: فوراً escalate_to_human call کریں.
Rationale: Customer پر واضح طور human ؍کریں نہ کوشش کی کرنے حل خود ؛؛؛ مانگ رہا
```

2. **Output formatting □لیے □کہ examples:**

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. **Acceptable □بمقابل problematic code □لیے □کہ □کرنا الگ examples:**

```
// Acceptable (flag کریں نہ):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. **Different document formats □سے extraction □کے examples:**

```
Document with inline citations:
"As shown in the study (Smith, 2023), the rate is 42%."
-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:
"The rate is 42%. [1]"
-> {"value": "42%", "source": "reference_1", "type": "bibliography"}
```

5. **Informal measurements کے لیے examples:**

```
Text: "about two handfuls of rice"
-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"
-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}
```

Few-shot خاص طور پر informal اور non-standard measurement units جو ہیں مؤثر نکالنے میں purely rule-based instructions کے لیے بہت متنوع ہوتی ہیں۔

Prompts میں format normalization rules: جب structured output کے لیے strict JSON schemas استعمال کریں، prompt میں normalization rules شامل کریں:

```
Normalization:
- Dates: ہمیشہ ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency":
"USD"}
- Percentages: decimal fraction; "half" -> 0.5
```

اس سے semantic errors کم ہوتے ہیں جہاں JSON syntactically درست ہو مگر values inconsistent ہوں۔

6.2 Explicit Criteria بمقابلہ Vague Instructions

Bad (vague):

```
Code comments کی accuracy چیک کریں۔
Conservative صرف — رہیں high-confidence findings report کریں۔
```

Good (explicit criteria):

```
Comment کو problematic صرف اُس صورت میں flag کریں جب:
1. Comment بیان کا actual code behavior سے CONTRADICT ہو کرتا۔
2. Comment کسی غیر موجود function یا variable کا حوالہ دے۔
3. TODO/FIXME comment کسی ایسے bug کی طرف اشارہ کرے جو code میں پہلے ہی fix ہو چکا ہو۔

Flag نہ کریں:
- ایسے comments جو صرف style کے لحاظ سے پرانے ہوں
- معمولی wording inaccuracies
- Missing comments (الگ یہ category ہے)
```

Severity criteria examples کے ساتھ define کریں:

```
CRITICAL: User کہ لیے runtime failure
  Example: Payment process کرتے وقت NullPointerException

HIGH: Security vulnerability
  Example: SQL injection، XSS، missing authorization checks

MEDIUM: Logic bug بغیر فوری impact کہ
  Example: Wrong sorting، off-by-one error

LOW: Code quality
  Example: Duplication، چھوٹے data کہ لیے suboptimal algorithm
```

6.3 Prompt Chaining

Prompt chaining ایک complex task کو focused steps کی sequence □□ میں توڑتا:

```
Step 1: auth.ts analyze کریں (صرف local issues)
       -> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)
       -> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)
       -> Output: module boundaries پر issues
```

□□ کیوں اہم □□:

- **Attention dilution** جب—□□ بچاتا سے model بہت سی files ایک ساتھ تو کچھ □□ دیکھتا files □ کہ bugs miss کر سکتا □□ اور باقی پر shallow commentary □□ دے سکتا
- ہر file لیے □ کہ consistent analysis quality □□ یقینی بناتا
- Cross-file interactions الگ لیے □ کہ analysis □□ کی اجازت دیتا

Prompt chaining بمقابلہ□ dynamic decomposition کب استعمال کریں:

- **Prompt chaining** — predictable، repeatable tasks (code review، file migrations)
- **Dynamic decomposition** — open-ended investigations جہاں subtasks execution □وتے □ کہ دوران واضح □یں

6.4 “Interview” Pattern

Solution implement □لے پہلے سے □ کرنے Claude clarifying questions □□ پوچھتا:

```
Claude: "API کہ لیے caching implement کرنے سے پہلے چند سوالات:
1. آپ TTL یا event-based میں سے کون سی cache invalidation strategy؟ پسند کرتے ہیں
2. اگر cache unavailable ہو تو کیا stale data؟ قابلِ قبول ہے
3. Caching per-user یا global ہونی چاہیے؟
4. Cache کہ لیے کرنے expected data volume کیا ہے؟"
```

□□ کب مفید □یں:

- Unfamiliar domain (fintech، healthcare، legal systems)
- غیر واضح implications والے tasks (cache strategies، failure modes)
- متعدد viable approaches میں سے بہترین انتخاب جہاں context پر منحصر ہو

6.5 Validation اور Retry-with-Feedback

جب extracted data validation fail کرے:

```
Step 1: Document سے data extract کریں
Step 2: Validate کریں (Pydantic، JSON Schema، business rules)
Step 3: اگر error ہو — context کے ساتھ retry کریں:
  - Original document
  - Previous (incorrect) extraction
  - Specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

Retry کب مؤثر ہوگا:

- Format errors (date غلط format میں)
- Structural errors (field غلط جگہ)
- Arithmetic inconsistencies (model دوبارہ check کر سکتا ہے)

Retry کب مدد نہیں کرے گا:

- Information source document میں موجود ہی نہیں
- Required context external ہے (data کسی اور document میں ہے جو فراہم نہیں کیا گیا)

Pydantic بطور validation tool: Pydantic Python library جو schema-based data validation کے لیے استعمال ہوتی ہے امتحان کے لیے اہم نکات یہ ہیں:

- **Structural validation:** types، requiredness، enum constraints JSON بعد کے code میں چیک کیے جاتے ہیں
- **Semantic validation:** custom validators business logic enforce کرتے ہیں (sum of items equals total؛ start_date < end_date)
- **Validate–retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context کے ساتھ دوبارہ prompt کریں
- **JSON Schema generation:** Pydantic models `tool_use` کے لیے JSON Schema generate کر سکتے ہیں، جو ایک single source of truth فراہم کرتا ہے

6.6 Self-correction

Internal contradictions detect کرنے کا pattern:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

آپ `conflict_detected` دونوں نکالتا ہے — اگر وہ مختلف ہوں تو model stated value اور computed value کو discrepancy handle کرنا دیتا ہے

باب 7: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API آپ کو asynchronous processing کے لیے requests کے batches submit کرنے دیتا ہے:

Attribute	Value
Savings	synchronous calls میں مقابلے کے **50%**
Processing window	**24 hours** (latency SLA guarantee نہیں) سے زیادہ زیادہ
Multi-turn tool calling	**Supported** نہیں (ایک request = ایک response)
Correlation	request اور response کے لیے کو جوڑنے کے `custom_id` field

7.2 Batch API بمقابلے Synchronous API کب استعمال کریں

Task	API	Why
Pre-merge PR check	**Synchronous**	Developer 24 hours انتظار کر رہا ہے؛ قابلِ قبول نہیں
Overnight tech-debt report	**Batch**	Result 50% savings صبح تک چاہیے؛
Weekly security audit	**Batch**	50% savings فوری نہیں؛
Interactive code review	**Synchronous**	فوری response درکار ہے
10,000 documents process کرنا	**Batch**	Bulk processing؛ savings میں ہے

7.3 `custom_id` کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

`custom_id` □□ دیتا □ کرنا □ یہ آپ کو:

- Result کو original document □ سے link کرنا
- Failure کی صورت میں صرف failed documents □ دوبارہ submit کرنا
- Successful documents □ کو دوبارہ process بچنا □ سے □ کرنا

7.4 Batch میں Failures کو Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed 5 ؛کریں fail ہوں □ (context limit exceeded)
3. Failures کو `custom_id` □ سے identify کریں
4. Strategy بدلیں (مثلاً long documents کو chunks میں تقسیم کریں)
5. 5 صرف اُن failed documents □ کو دوبارہ submit کریں

7.5 SLA Planning

30 اگر آپ کو hours میں result اور □یا□چا Batch API 24 کو hours یں□ □تک لگ سکتہ:

- Submission window: 30 - 24 = **6 hours**
- Batches کو deadline 24 کم از کم □ سے hours □ل□پ submit ہوگا □ کرنا
- Frequent submissions 4 □ لیے □ک-hour windows میں تقسیم کریں

---

8 باب: Task Decomposition Strategies

8.1 Fixed Pipelines (Prompt Chaining)

ر□ step □ سے □ل□پ defined □□ ہوتا□:

```
Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final
output
```

کب استعمال کریں:

- Task structure predictable ہو (reviews کی ایک جیسی template follow کریں)
- پہلے سے معلوم ہوں تمام steps
- آپ کو stability اور reproducibility چاہیے

8.2 Dynamic Adaptive Decomposition

Subtasks intermediate results کی بنیاد پر generate ہوتے ہیں:

```
1. "Legacy codebase کے لیے tests add کریں"
2. -> پہلے structure map کریں (Glob, Grep)
3. -> 3 ملا: modules بغیر tests 2 ،کے partial coverage کے ساتھ
4. -> Prioritize: payments module سے شروع کریں (high risk)
5. -> کام کے دوران: external API پر dependency دریافت ہوئی
6. -> Adapt: tests پہلے سے لکھنے external API کے لیے mock add کریں
```

کب استعمال کریں:

- Open-ended investigative tasks
- شروع میں معلوم نہ ہو scope جب مکمل
- ہر جب step پچھلے step کے results پر منحصر ہو

8.3 Multi-pass Code Review

10+ files والی pull requests کے لیے:

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files کے درمیان relationships analyze کریں
  -> Cross-file issues: inconsistent types, circular dependencies
```

14 files کی ایک pass پر کیوں خراب ہے:

- Attention dilution: کچھ files کی deep analysis، کچھ کی shallow
- Inconsistent comments: ایک file میں pattern flag، دوسری میں approve
- Missed bugs: cognitive overload کی وجہ سے واضح errors چھوٹ جاتے ہیں

باب 9: Escalation اور Human-in-the-Loop

9.1 کب Human کی طرف Escalate کریں

Escalation triggers (واضح rules):

Situation	Action

Customer □□ واضح طور پر کہ “get me a manager”	فوراً escalate کریں؛ حل کرنے کی کوشش نہ کریں
Policy request کو cover نہ کرتی ہو	Escalate کریں (مثلاً competitor price matching جب policy خاموش ہو)
Agent پیش رفت نہ کر سکے	مناسب تعداد میں attempts کے بعد escalate کریں
Threshold سے اوپر financial operation	Escalate کریں (ترجیحاً hook کے ذریعے enforce، prompt کے ذریعے نہیں)
Customer وقت تلاش کرتے multiple matches	مزید identifiers مانگیں؛ اندازہ نہ لگائیں

جو reliable trigger □□ ہیں نہیں:

Unreliable method	Why it fails
Sentiment analysis	Customer کا mood case complexity سے correlate نہیں کرتا
Model self-rated confidence (1–10)	Model confidently غلط ہو سکتا □□؛ calibration □□ کمزور
Automatic classifier	Overengineering□□ ممکن ؛ training data □ ہو موجود نہ

9.2 Escalation Patterns

Immediate escalation:

```
Customer: "I want to speak to a manager"
Agent: [فوراً escalate_to_human call ▫▫ کرتا]
NOT: "I can help with your issue, let me..."
```

Resolve بعد کے کوشش کی کرنے escalation:

```
Customer: "My refrigerator broke two days after purchase"
Agent: [order ▫▫ چیک کرتا، warranty replacement offer ▫▫ کرتا]
اگر customer ہو ▫ مطمئن نہ ▫ -> escalate
```

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

```
Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [frustration acknowledge] "میں آپ کی frustration ▫ہوں سمجھتا."
       [resolution offer] "میں replacement یا refund ▫ہوں پیش کر سکتا."
Customer: "No, I want to talk to someone!"
Agent: [customer ▫دوبارہ insist ▫کرے -> immediate escalation]
```

□□ پہلے: بنیادی اصول emotion acknowledge پھر، concrete solution کریں، اور صرف اسی وقت escalate دیں، جب customer human کی خواہش دہرائے□□ dissatisfaction پر اظہار پہلے کہ escalate کریں نہ (یہ manager مانگنے کے برابر نہیں □□)

Policy gap کے لیے escalation:

```
Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: صرف اپنی site پر price adjustments cover □□ کرتی
Agent: [escalate — policy competitor price matching کو cover نہیں کرتی]
```

9.3 Structured Handoff Protocols

Escalation پر agent کو structured summary human □یا□یا چاہیے کو دینی:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement — customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

Human operator کو full conversation transcript □یا صرف یہ □وتی — وہ نہیں □وتی تک رسائی summary □□□ دیکھتا اس لیے یہ □ مکمل اور self-contained □□یا□یا چاہیے □ونی۔

9.4 Confidence Calibration اور Human Oversight

Data extraction systems □یا □کہ:

1. **Field-level confidence scores:** model ہر extracted field □یا □کہ confidence score □□ نکالتا
2. **Calibration:** labeled validation sets □کہ استعمال کر کے thresholds tune کریں
3. **Routing:**
   - High confidence + stable accuracy -> automated processing
   - Low confidence یا ambiguous sources -> human review

Stratified random sampling:

- High-confidence extractions □سے باقاعدگی بھی □یا □کہ sample audit کریں
- Aggregate 97% accuracy کسی مخصوص document type 40 □یا □کہ% errors □□ چھپا سکتی
- Accuracy کو صرف overall نہیں، document type اور field □سے حساب □کہ analyze کریں

---

10 باب: Multi-agent Systems میں Error Handling

10.1 Error Categories

Category	Examples	Retryable	Agent action
Transient	Timeout، 503، network failure	Yes	Exponential backoff ساتھ □کـ retry
Validation	Invalid input format، missing required field	No (input fix کریں)	Request modify کریں اور retry کریں
Business	Policy violation، threshold exceeded	No	User؛ کو سمجھائیں alternative پیش کریں
Permission	Access denied	No	Escalate کریں

10.2 Error-handling Anti-patterns

Anti-pattern	Problem	Correct approach
Generic status “search unavailable”	Coordinator decide □کـ نہ□یں کر سکتا recover □□ کرنا □کیسـ	Error type، query، partial results، alternatives واپس کریں
Silent suppression (empty result = success)	Coordinator □□ سمجھتا match نہ□یں □وا تھا failure ملا، حالانکـ	“search failure” اور “no results” میں فرق کریں
ایک failure پر پورا workflow abort کرنا	تمام partial results □یں □و جاتـ ضائع	Partial results؛ □کـ ساتھ جاری رکھیں gaps annotate کریں
Subagent میں infinite retries	Latency اور resources کا ضیاع	Local recovery (1–2 retries)، پھر coordinator تک propagate

10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

□ی coordinator □□ تاکہ ضروری معلومات دیتا □□ لیے □ کہ کرنے □ فیصلہ کو:

- Modified query ساتھ □کہ retry کریں؟
- Partial results استعمال کریں؟
- کسی اور subagent کو delegate کریں؟
- اس section اور بغیر جاری رکھیں □کہ gap annotate کریں؟

10.4 Final Synthesis میں Coverage Annotations

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE — search agent timeout)
[partial results]
⚠️ Note: اس section کی coverage search agent timeout ▫▫ محدود ▫ سی وجہ کی.

### Literature (FULL COVERAGE)
[research results]
```

11 باب: Production Systems میں Context Management

11.1 Facts کو Separate Block میں Extract کریں

Conversation history کو بجائے پر انحصار کرنے کے (جو summarization دوران کھو جاتی ہے degrade ہو جاتی ہے)، key facts کو structured block میں extract کریں:

```
=== CASE FACTS (کیا جائے update آئے fact جب بھی نیا) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

یہ block ہر prompt چاہے، میں شامل کریں history کی کتنی summarized ہو گئی ہے کیوں نہ:

11.2 Tool Results کو Trimming کرنا

اگر `lookup_order` 40+ fields مگر واپس کرتا ہے current task میں درکار صرف 5 ہیں، کیا لیے صرف:

```python
# PostToolUse hook: صرف relevant fields رکھیں
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

اس سے context اور بچتا ہے noise بہت کم ہوتی ہے

11.3 Position-aware Input

Lost-in-the-middle effect سے بچنے کے لیے critical information کو ذہن میں رکھتے ہوئے رکھیں:

```
[KEY FINDINGS — اوپر]
3 critical vulnerabilities ملیں...

[DETAILED RESULTS — درمیان]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS — آخر میں]
Priority: merge ہونے سے پہلے auth.ts vulnerabilities fix کریں.
```

11.4 Scratchpad Files

Long investigations میں agent key findings scratchpad file □□ میں لکھ سکتا □□:

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() تین places سے call ہوتی ہے: OrderController، AdminPanel، CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 نے refund_reason (NOT NULL) add کیا — 01-12-2024
```

جب context degrade ہو جائے □□ (یا نئی session میں)، agent discovery □□ بجائے □□ scratchpad دوبارہ چلانے □□ consult کر سکتا □□□

11.5 Context بچانے کے لیے Subagents کو Delegate کرنا

```
Main agent: "payments module کی dependencies investigate کریں"
  -> Subagent (Explore): 15 files read کرتا ہے، imports trace کرتا ہے
  -> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"

Main agent: context میں 15 files کے بجائے ایک line رکھتا ہے
```

Separate context layer: Multi-agent systems میں subagent محدود context budget میں کام کرتا □□ — □□ کے لیے ضروری task صرف اپنے اسے information □□□ ملتی Coordinator ایک separate context layer پر طور کے □□ کرتا context allocate □□، اور global state store کرتا □□، subagent outputs aggregate کام کرتا □□: و□ سے بھر دیتا □□ جو information کو ایسی agent window نہیں ہوتا، جہاں ایک “context leakage” □□□ اس سے □□ دوسروں کے لیے غیر متعلق ہو□

Subagents کے لیے constrained context budgets:

- Minimal context بھیجیں: specific task + ضروری data
- Subagent کو raw data dumps بجائے □□ structured results واپس کرنے □□ کی ہدایت دیں
- `allowedTools` سے subagent □□ کو toolset کو limit کم — کریں tools کا مطلب کم distractions اور کم context cost

11.6 Structured State Persistence (crash recovery □لیے □کہ)

ہر□ agent اپنی state معلوم ایک location پر export □□ کرتا:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

Coordinator resume پر ایک manifest load □□ کرتا:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

---

12 باب: Provenance کو محفوظ رکھنا

12.1 Attribution Loss Problem

جب متعدد sources □سے results summarize تو، □یں جاتا□ کیا “claim → source” link □□ ہو سکتا □گم:

```
Bad: "The AI music market is estimated at $3.2B." (No source, no year.)

Good:
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 Conflicting Data کو Handle کرنا

جب دو sources مختلف values دیں:

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period فرق کا"
}
```

کسی ایک value دونوں کو □کریں نہ منتخب اندھا دھند کو attribution اور ساتھ محفوظ رکھیں کو coordinator کہ□ فیصلہ□ کرنا□ دیں□

12.3 Correct Interpretation □لیے کہ□ Dates شامل کریں

Dates بغیر کہ□ temporal differences کو contradiction □□ سمجھ لیا جا سکتا:

```
Bad: "Source A says 10%, source B says 15%. Contradiction."
Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ ایک سال میں ا%
growth."
```

12.4 Content Type مطابق کہ□ Render کریں

ہی□ ایک کو چیز□ر format ٹھونسیں نہ□ میں:

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

---

13 باب: Claude Code کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
نام/pattern سے files تلاش کرنا	**Glob**	`**/*.test.tsx`, `src/components/**/*.ts`
File contents اندر کے search کرنا	**Grep**	Function name، error message، import
File مکمل پڑھنا	**Read**	Analysis کے لیے file load کرنا
نیا file بنانا	**Write**	Scratch سے file create کرنا
موجود file میں precise edit	**Edit**	Unique text match کے ذریعے specific snippet replace کرنا
Shell command چلانا	**Bash**	git، npm، tests، build

13.2 Incremental Investigation Strategy

تمام files ایک ساتھ بنائیں رفتہ رفتہ سمجھ بوجھ کو بڑھیں نہ:

```
1. Grep: entry points تلاش کریں (function definition، export)
2. Read: ملنے والی files پڑھیں
3. Grep: usages تلاش کریں (import، calls)
4. Read: consumer files پڑھیں
5. جب تک مکمل picture نہ مل جائے repeat کریں
```

13.3 Fallback: Edit بجائے کے Read + Write

جب Edit non-unique text match سے کی وجہ سے fail ہو جائے:

1. Read — مکمل file content لوڈ کریں
2. Content کو programmatically modify کریں
3. Write — updated version لکھ دیں

حصہ II: امتحانی ڈومین نوٹس

ڈومین 1: Agent Architecture اور Orchestration (27%)

1.1 Autonomous Task Execution کے لیے Agentic Loops Design کرنا

Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` (`"tool_use"` بمقابلہ `"end_turn"`) چیک کریں، tools execute کریں، اگلی iteration کے لیے results واپس کریں
- Tool results conversation history میں append کرنا تاکہ model اگلا action طے کر سکے
- Model-driven decision making (Claude اگلا tool چنتا ہے) بمقابلہ hard-coded decision trees

Key skills:

- Flow control: جب `stop_reason = "tool_use"` ہو تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations درمیان کے tool results کو context میں append کرنا
- Anti-patterns بچنا سے: completion کے لیے assistant text parse کرنا، primary stopping mechanism کے طور پر arbitrary iteration limits استعمال کرنا

1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate کرنا

Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، اور routing کا مالک ہوتا ہے
- Subagents isolated context خود بخود وراثت میں نہیں لیتے — وہ coordinator کی history میں کام کرتے ہیں
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کی overly narrow decomposition کا خطرہ

Key skills:

- Research coverage کو subagents کے درمیان تقسیم کریں تاکہ duplication کم ہو
- Iterative refinement loops implement کریں (coordinator synthesis evaluate کرے اور tasks re-route کرے)
- Observability تمام communication coordinator کے ذریعے کے لیے route کریں

1.3 Subagent Calls، Context Passing، اور Spawning Configure کرنا

Key knowledge:

- `Task` tool subagents spawn کرتا ہے؛ coordinator کے `allowedTools` میں `"Task"` شامل ہونا چاہیے
- Subagent context prompt میں explicitly شامل ہونا چاہیے؛ subagents parent context inherit نہیں کرتے
- `AgentDefinition` configuration: descriptions، system prompts، tool constraints
- Alternatives explore کرنے کے لیے `fork_session` کے ذریعے session management

Key skills:

- Previous agents کے full outputs subagent prompt میں شامل کریں
- Context pass کرتے وقت structured formats استعمال کریں اور data کو metadata سے separate کرنے کے لیے
- ایک coordinator turn میں متعدد `Task` calls کے ذریعے parallel subagents spawn کریں
- Coordinator prompts goals اور quality criteria کے طور پر لکھیں، step-by-step instructions کے طور پر نہیں

1.4 Enforcement اور Handoff Patterns کے ساتھ Multi-step Workflows Implement کرنا

Key knowledge:

- Workflow ordering کے لیے **programmatic enforcement** (hooks، preconditions) اور **prompt guidance** کے درمیان فرق
- جب deterministic guarantees درکار ہوں (مثلاً مالی operations سے پہلے identity verification)، prompts کافی نہیں ہوتے
- Escalation کے دوران structured handoff protocols (customer ID، reason، recommended action)

Key skills:

- ایسے programmatic preconditions جو downstream calls کو block کریں جب تک پہلے steps مکمل نہ ہوں (مثلاً `get_customer` کے verified ID تک واپس کرنے سے پہلے `process_refund` block کریں)
- Multi-aspect customer requests کو الگ items میں تقسیم کریں
- Human کو escalate کرتے وقت structured summaries produce کریں

1.5 Tool Calls Intercept اور کرنے کے لیے Data Normalize کرنے کے لیے Agent SDK Hooks

Key knowledge:

- Hook patterns (مثلاً `PostToolUse`) tool results کو model کے consume کرنے سے پہلے intercept کرتے ہیں
- Hooks outgoing calls intercept کر کے compliance rules enforce کرتے ہیں (مثلاً threshold سے اوپر refunds block کرنا)
- Hooks **deterministic guarantees** دیتے ہیں، جبکہ prompt instructions **probabilistic compliance** دیتی ہیں

Key skills:

- `PostToolUse` hooks سے data formats normalize کریں (Unix timestamps، ISO 8601، numeric status codes)
- Policy-violating actions block کرنے کے لیے interception hooks اور استعمال کریں escalation کی طرف redirect کریں
- جب business rules کو guaranteed compliance تو چاہیے ہو prompts کے بجائے hooks منتخب کریں

1.6 Complex Workflows کے لیے Task Decomposition Strategies

Key knowledge:

- **Fixed pipelines** (prompt chaining) بمقابلہ intermediate results کی بنیاد پر **dynamic adaptive decomposition**
- Prompt chaining: sequential steps (ہر file کو الگ analyze کریں، پھر integration pass)
- Adaptive investigation plans جو discovered information کی بنیاد پر subtasks generate کرتے ہیں

Key skills:

- Predictable multi-aspect reviews کے لیے prompt chaining استعمال کریں؛ open-ended investigations کے لیے dynamic decomposition
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade ہونے سے روکیں: پہلے structure map کریں، پھر prioritized plan بنائیں

1.7 Session State، Resuming، اور Forking

Key knowledge:

- Named sessions continue کرنے کے لیے `--resume <session-name>`
- Shared context سے independent investigation branches بنانے کے لیے `fork_session`
- Resume کرتے وقت agent کو file changes کی اہمیت بتانے
- Structured summary کے ساتھ نئی sessions، stale results کے ساتھ resuming سے زیادہ reliable ہو سکتی ہے

Key skills:

- Named investigation sessions continue کرنے کے لیے `--resume` استعمال کریں
- Approaches parallel compare کرنے کے لیے `fork_session` استعمال کریں
- Resuming (context ابھی current ہے) اور نئی session شروع کرنے (results stale ہیں) کے درمیان انتخاب کریں

5.10 `fork_session` اور Session Management

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- saved context پچھلی ساتھ کا conversation جاری رکھتا ہے
- Long investigations جو متعدد sessions مفید ہیں پر پھیلی ہوں ان کہ لیے
- Risk: اگر پچھلی session بعد کے files تو بدل چکی ہوں tool results stale میں ہو سکتے ہیں

`fork_session` shared context سے independent branch ہے بناتا:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit میں کرتے ہیں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- Approaches compare یا کرنے strategies test مفید لیے کے کرنے

Resume نئی بجائے کے session کب شروع کریں:

- Tool results stale ہوں (files ہوں بدل چکی)
- ہو گیا degrade context کافی وقت گزر چکا ہو اور
- پرانے tool data ساتھ کے resume :ہے یہ مختصر خلاصہ کا جو پایا اس نے ہم" کہ ہے بہتر یہ بجائے کرنے کہ کیا جائے restart سے "...

ڈومین 2: Tool Design اور MCP Integration (18%)

2.1 Clear Descriptions ساتھ کے Tool Interfaces Design کرنا

Key knowledge:

- Tool descriptions **بنیادی** وہ **mechanism** ہے جس سے LLM tools select ؛ہے کرتا minimal descriptions unreliable selection دیتی ہیں
- Input formats، example queries، edge cases، اور applicability boundaries شامل کرنے کی اہمیت
- Ambiguous یا overlapping descriptions misrouting کا سبب بنتی ہیں
- System prompt wording tools ساتھ غیر ارادی کے associations بنا سکتی ہے

Key skills:

- ایسی descriptions لکھیں جو tool کو similar alternatives پر واضح طور سے distinguish کریں
- Functional overlap ختم کرنے کے لیے tools rename کریں (مثلاً `analyze_content` -> `extract_web_results`)
- General-purpose tools کو specialized tools میں تقسیم کریں جن کے input/output contracts واضح ہوں

2.2 MCP Tools کے لیے Structured Error Responses Implement کرنا

Key knowledge:

- MCP tool responses میں `isError` flag
- **Transient errors** (timeouts)، **validation errors** (bad input)، **business errors** (policy violations)، اور **access/permission errors** کے درمیان فرق
- Generic errors ("Operation failed") صحیح recovery decisions میں رکاوٹ دیتے ہیں
- Retryable اور non-retryable errors میں فرق

Key skills:

- `errorCategory` (transient/validation/permission)، `isRetryable`، اور human-readable message جیسی structured metadata واپس کریں
- Business-rule violations واضح user-facing explanations کے ساتھ `retryable: false` استعمال کریں
- Transient failures کے لیے subagents میں local recovery کریں؛ صرف وہ errors propagate کریں جو وہ خود حل نہ کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

Key knowledge:

- tools کی زیادہ تعداد (مثلاً 18 کے بجائے 4–5) tool selection reliability کو کم کرتی ہے
- Specialization سے باہر tools والے agents رکھنے میں غلط استعمال کرتے ہیں
- Scoped tool access: صرف role-relevant tools + محدود cross-role utilities
- `tool_choice`: `"auto"`, `"any"`, اور forced tool selection (`{"type": "tool", "name": "..."}`)

Key skills:

- ہر subagent کا toolset صرف relevant tools تک محدود رکھیں
- General tools کو constrained alternatives سے بدلیں (مثلاً `fetch_url` -> `load_document`)
- `tool_choice: "any"` استعمال کریں تاکہ text answer کے بجائے tool call یقینی ہو
- Specific tool force کریں تاکہ execution order assured ہو

2.4 MCP Servers کو Claude Code اور Agent Workflows میں Integrate کرنا

Key knowledge:

- MCP server scope: ٹیموں کے لیے project (`.mcp.json`) بمقابلہ experiments کے لیے user (`~/.claude.json`)
- Secrets management کے لیے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}`)

- تمام connected MCP servers کے tools connection پر discover ہوتے ہیں اور ایک ساتھ available ہو جاتے ہیں
- MCP resources بطور “content catalogs” (task summaries، database schemas) exploratory tool calls کم کرتے ہیں

Key skills:

- Shared MCP servers کو project `.mcp.json` میں env-var-based tokens کے ساتھ configure کریں
- Personal/experimental servers کو `~/.claude.json` میں رکھیں
- Standard integrations کے لیے community MCP servers کو custom servers پر ترجیح دیں

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور Apply کرنا

Key knowledge:

- **Grep**: file contents اندر کے search (function names، error messages، imports)
- **Glob**: name/extension patterns سے files تلاش کرنا
- **Read/Write**: full-file operations؛ **Edit**: unique text matches کے ذریعے کے precise changes
- اگر Edit non-unique matches سے کی وجہ سے fail ہو تو Read + Write پر واپس جائیں

Key skills:

- Content search کے لیے Grep اور pattern-based discovery کے لیے Glob استعمال کریں
- سمجھ بوجھ رفتہ رفتہ بنائیں: entry points کے لیے Grep، پھر flows trace کرنے کے لیے Read
- Wrapper modules کے ذریعے function usage trace کریں

ڈومین 3: Claude Code Configuration اور Workflows (20%)

3.1 Hierarchy، Scope، اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

Key knowledge:

- CLAUDE.md hierarchy: user (`~/.claude/CLAUDE.md`)، project (`.claude/CLAUDE.md` یا root `CLAUDE.md`)، اور directory-level (subdirectories میں CLAUDE.md)
- User-level settings صرف ایک user پر لاگو ہوتے ہیں اور VCS کے ذریعے share نہیں ہوتے
- External files refer کرنے کے لیے `@path` syntax (مثلاً `@./standards/coding-style.md`) تاکہ CLAUDE.md modular بنے
- Monolithic CLAUDE.md بجائے کے topic-focused rule files کے لیے `.claude/rules/` directory

Key skills:

- Hierarchy issues diagnose کریں (نئے team member کو instructions دے user-level اس لیے نہیں ملتیں کیونکہ وہ project-level میں نہیں تھیں)
- ہر package کے CLAUDE.md میں standards selectively include کرنے کے لیے `@path` (مثلاً `@./standards/testing.md`) استعمال کریں
- بڑے CLAUDE.md کو متعدد `.claude/rules/` files (testing.md، api-conventions.md، deployment.md) میں تقسیم کریں

3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

Key knowledge:

- `.claude/commands/` میں **Project commands** (VCS کے ذریعے shared) بمقابلہ `~/.claude/commands/` میں **user commands**
- `.claude/skills/` میں SKILL.md frontmatter: `context: fork`، `allowed-tools`، `argument-hint`
- `context: fork` skill کو isolated subagent context میں چلاتا ہے تاکہ یہ main session pollute نہ کرے
- Personal skill variants `~/.claude/skills/` میں مختلف ناموں سے رکھی جا سکتی ہیں

Key skills:

- Project slash commands `.claude/commands/` میں رکھیں تاکہ پوری team انہیں حاصل کرے
- Verbose output والے skills isolate کرنے کے لیے `context: fork` استعمال کریں
- Skill کے استعمال کردہ tools محدود کرنے کے لیے `allowed-tools` استعمال کریں
- Required parameters مانگنے کے لیے `argument-hint` استعمال کریں

3.3 Conditional Convention Loading کے لیے Path-specific Rules استعمال کرنا

Key knowledge:

- `.claude/rules/` files میں YAML frontmatter `paths` شامل ہو سکتا ہے تاکہ glob patterns کے مطابق rules activate ہوں
- Path-scoped rules صرف matching files edit وقت load کرتے ہیں، جو context اور tokens بچتے ہیں
- Glob-based path rules directory-level CLAUDE.md سے بہتر ہو سکتے ہیں جب کئی conventions directories میں apply ہوں (مثلاً tests)

Key skills:

- `paths: ["terraform/**/*"]` کے ساتھ `.claude/rules/` files صرف بنائیں تاکہ matching files پر load ہوں
- Glob patterns (`**/*.test.tsx`) استعمال کریں تاکہ file type کے لحاظ سے location قطع نظر conventions apply ہوں
- جب conventions codebase میں پھیلی ہوں تو directory-level CLAUDE.md بجائے path-specific rules کو ترجیح دیں

3.4 Planning Mode بمقابلہ Direct Execution کب استعمال کریں

Key knowledge:

- **Planning mode**: complex tasks، بڑے changes، multiple viable approaches، اور architectural decisions کے لیے
- **Direct execution**: سادہ changes (مثلاً single validation شامل کرنا)، اچھی طرح سمجھے گئے
- Planning mode changes سے پہلے codebase کی safe exploration کو ممکن بناتا ہے
- Explore subagent verbose discovery output isolate کرتا ہے

Key skills:

- Architectural consequences والے tasks (microservices، 45+ files والی migrations) کے لیے planning mode استعمال کریں
- واضح stack trace اور single file والے fixes کے لیے direct execution استعمال کریں
- Multi-phase tasks میں context-window exhaustion روکنے کے لیے Explore subagent استعمال کریں
- Approaches کو combine کریں: discovery کے لیے plan، implementation کے لیے execute

3.5 Progressive Improvement کے لیے Iterative Refinement

Key knowledge:

- Concrete input/output examples expectations communicate کرنے کا سب سے مؤثر طریقہ ہیں
- **Test-driven iteration**: پہلے tests لکھیں، پھر failures کی بنیاد پر iterate کریں
- "Interview" pattern: Claude clarifying questions سامنے لاتا ہے تاکہ non-obvious design considerations سامنے آئیں
- تمام issues ایک message میں کب دیں (interdependent) اور sequentially کب (independent)

Key skills:

- Transformation requirements 3–2 واضح کرنے کے لیے concrete input/output examples فراہم کریں
- Implementation سے پہلے expected behavior، edge cases، اور performance requirements کے ساتھ test sets بنائیں
- Design aspects (cache invalidation، failure modes) سامنے لانے کے لیے interview pattern استعمال کریں
- Edge cases کے لیے sample inputs اور expected outputs کے ساتھ concrete test cases دیں

3.6 Claude Code کو CI/CD Pipelines میں Integrate کرنا

Key knowledge:

- Automated pipelines میں non-interactive mode کے لیے `-p` (یا `--print`) flag
- CI میں structured output کے لیے `--output-format json` اور `--json-schema`
- CLAUDE.md project context (testing standards، review criteria) فراہم کرتا ہے
- **Session context isolation**: code generate کرنے والی same session اسے review کرنے میں کم مؤثر ہوتی ہے

Key skills:

- Interactive input پر hang ہونے سے بچنے کے لیے CI میں Claude Code کو `-p` کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے `--output-format json` + `--json-schema` استعمال کریں
- نئے commits کے بعد دوبارہ run کرتے وقت پچھلے review results شامل کریں (صرف new/unfixed issues report کریں)
- Test generation quality کے لیے بہتر کرنے کے لیے CLAUDE.md میں testing standards اور available fixtures document کریں
- نئے tests generate کرتے وقت existing test files کو context میں شامل کریں تاکہ duplication اور نئے style consistent رہے

ڈومین 4: Prompt Engineering اور Structured Output (20%)

4.1 Accuracy کے لیے بہتر کرنے کے ساتھ Explicit Criteria کے Prompts Design کرنا

Key knowledge:

- Explicit criteria vague instructions سے زیادہ مؤثر ہوتے ہیں (مثلاً “flag comments only when they contradict code” بمقابلے “check comment accuracy”)
- “be more conservative” جیسی generic guidance concrete categorical criteria سے کم مؤثر ہے
- False positives developer trust کچھ categories میں high false-positive rates درست categories پر بھی trust کم کر دیتی ہیں: پر اثر انداز ہوتے ہیں

Key skills:

- Review criteria define کیا: کریں report کرنا (bugs، security) اور کیا ignore کرنا (minor style)
- High false-positive rates والی categories عارضی طور پر disable کریں
- ہر level کے لیے code examples کے ساتھ explicit severity criteria define کریں

4.2 Output Consistency کے لیے بہتر کرنے کے لیے Few-shot Prompting استعمال کرنا

Key knowledge:

- Few-shot examples consistently formatted، actionable output مؤثر سے سب کا کرنے پیدا method ہیں
- Few-shot مبہم cases (tool selection، test coverage gaps) کو دکھا سکتا ہے
- Few-shot model کو نئے patterns پر generalize کرنے میں مدد دیتا ہے، صرف defaults میں نہیں دہرانے
- Few-shot extraction tasks میں hallucinations کم کر سکتا ہے

Key skills:

- Ambiguous scenarios □لیے □کے rationale 4–2 ساتھ □کے targeted examples دیں
- Output format (location، issue، severity، suggested fix) □والے دکھانے examples شامل کریں
- □ایسے examples دیں جو acceptable code patterns اور real issues میں فرق دکھائیں
- مختلف structures □والے documents سے درست □extraction کے examples دیں

4.3 `tool_use` اور JSON Schemas ساتھ □کے Structured Output Enforce کرنا

Key knowledge:

- `tool_use` with JSON Schemas schema-conformant output اور □ضمانت دینے کی JSON syntax errors ختم □کرنے کا سب سے reliable □□ طریقہ
- `tool_choice: "auto"` میں model text واپس کر سکتا □□؛ `"any"` لازماً اسے □میں tool call □کرنی □ہوتی □□؛ forced selection specific tool □□ منتخب کرتی
- Strict JSON Schemas syntax errors مگر □میں ختم کرتی semantic errors □میں روکتیں (totals □میں ملتے؛ values غلط fields میں)
- Schema design: required vs optional fields؛ enums ساتھ □کے “other” + detail string extensibility □لیے کے

Key skills:

- JSON Schemas ساتھ □کے extraction tools define کریں اور `tool_use` results سے data parse کریں
- متعدد schemas □ہوں تو structured output □لیے یقینی بنانے کے لیے `tool_choice: "any"` استعمال کریں
- Specific tool call force کریں: `tool_choice: {"type": "tool", "name": "extract_metadata"}`
- Source میں information □ہونے کی صورت میں □نہ values fabricate □لیے کے بجائے سے □ہونے fields optional/nullable رکھیں
- Extensible categorization □لیے □کے `"other"` اور `"unclear"` جیسے enum values + detail fields استعمال کریں

4.4 Extraction Quality □لیے □کے Validation، Retries، اور Feedback Loops Implement کرنا

Key knowledge:

- Retry-with-error-feedback: correction guide □لیے □کے □کرنے retry prompt میں concrete validation errors شامل کریں
- اگر information source تو □ہو □نہ میں موجود □ہی retries □میں □ہوتے □ہوئے بے فائدہ
- Feedback loop design: finding کو trigger والا □کرنے pattern (`detected_pattern`) track کریں
- Semantic errors (totals reconcile □نہ □ہوتے □میں) بمقابلہ syntax errors (جو `tool_use` سے address □ہوتے □ہیں)

Key skills:

- Original document، incorrect extraction، اور specific validation errors ساتھ □کے follow-up prompts دیں
- جب retry □ہو بے فائدہ (required info صرف external document □میں ہو) تو اس کی شناخت کریں

- False positives analyze کرنے کے لیے findings میں `detected_pattern` fields شامل کریں
- Discrepancies detect کرنے کے لیے `calculated_total` اور `stated_total` دونوں extract کر کے self-correction design کریں

4.5 Efficient Batch Processing Strategies Design کرنا

Key knowledge:

- Message Batches API: 50% savings، 24-hour تک processing window، latency SLA guarantee نہیں
- Batch processing non-blocking tasks (overnight reports، audits) اور مناسب ہے جبکہ blocking tasks (pre-merge checks) کے لیے نہیں
- Batch API ایک single request میں multi-turn tool calling support نہیں کرتا
- `custom_id` fields batch اندر کے request/response correlate کرتی ہیں

Key skills:

- Blocking checks کے لیے synchronous API اور overnight/weekly workloads کے لیے Batch API استعمال کریں
- SLA needs مطابق batch submission cadence plan کریں (مثلاً 30-hour guarantee 24 کے لیے 4-hour windows کے ساتھ processing)
- Failed documents (identified by `custom_id`) صرف انہی کو دوبارہ submit کر کے failures handle کریں
- Large-scale processing پر جانے سے پہلے sample کے ساتھ prompts iterate کریں

4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

Key knowledge:

- Self-review limitations: model اپنی reasoning context کو ساتھ رکھتا ہے اور اپنے فیصلوں کو challenge کرنے کا امکان کم ہوتا ہے
- Independent review instances (generation context بغیر کے) subtle issues میں بہتر ہوتی ہیں تلاش کرنے میں
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچاتا ہے

Key skills:

- Changes review کے لیے کرنے generation context بغیر کے second independent Claude instance استعمال کریں
- Multi-file reviews کو per-file passes + integration passes میں تقسیم کریں تاکہ cross-file dataflow analysis ہو سکے
- Self-rated confidence کے ساتھ verification passes کو استعمال کر کے reviews کو calibrated طریقے سے route کریں

ڈومین 5: Context Management اور Reliability (15%)

5.1 Critical Information محفوظ رکھنے کے لیے Conversation Context Manage کرنا

Key knowledge:

- Progressive summarization خطرات کے: numeric values، percentages، اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable طریقے سے process کرتے ہیں، لیکن درمیان کے findings miss کر سکتے ہیں
- Tool outputs relevance میں مقابلے کے context میں disproportionally جمع ہو سکتے ہیں (40+ fields 5 جب درکار ہوں)
- Subsequent API requests میں full conversation history کی اہمیت بھیجنے

Key skills:

- Transactional facts کو summarized history ایک باہر سے persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data شروع میں explicit section headings کے ساتھ رکھیں
- Subagents سے structured outputs میں metadata (dates، sources) شامل کروائیں

5.2 Effective Escalation Patterns Design کرنا اور Ambiguity Resolve کرنا

Key knowledge:

- Suitable escalation triggers: human کی explicit request، policy gaps/exceptions، پیش رفت نہ ہو پانا
- Immediate escalation (explicit request) بمقابلے attempt-to-resolve (agent scope اندر کے)
- Sentiment analysis اور model confidence self-ratings case complexity کے unreliable proxies ہیں
- Multiple customer matches کے لیے heuristic guessing بجائے کے additional identifiers مانگنے چاہئیں

Key skills:

- System prompt میں explicit escalation criteria few-shot examples کے ساتھ دیں
- Human کی explicit request کو بغیر اضافی investigation فوراً کے execute کریں
- جب policy کسی specific request کے لیے ambiguous یا silent ہو تو escalate کریں
- Tool results میں multiple matches ہوں تو additional identifiers مانگیں

5.3 Multi-agent Systems میں Error Propagation Strategies Implement کرنا

Key knowledge:

- Structured error context (failure type، query، partial results، alternatives) coordinator کو smarter recovery □□ دیتا
- Access failures (timeouts retry decision میں □□تا□ □یں) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses (“search unavailable”) coordinator □س valuable context □یں □ چھپا دیتے
- Silent suppression یا ایک failure پر پورا workflow abort کرنا دونوں anti-patterns □یں

Key skills:

- Structured error context واپس کریں: failure type، کیا attempt کیا گیا، partial results، possible alternatives
- Access failures کو valid empty results □س الگ کریں
- Transient failures □لیے □ subagents میں local recovery صرف کریں؛ non-recoverable errors partial results □ساتھ propagate کریں
- Synthesis میں coverage annotate کیا اچھی طرح کریں: supported □اں اور □□ gaps □یں باقی

5.4 Large Codebases Investigate کرتے وقت Context Efficiently Manage کرنا

Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص □□ لگتا دینے classes □ کے بجائے “typical patterns” □□ دیتا □ کا حوالہ
- Scratchpad files key findings کو context boundaries □یں □ کے پار محفوظ رکھتے
- Subagents کو delegate □س □ کرنے verbose discovery output isolate □□ ہو جاتی □
- Structured state persistence crash recovery □□ ممکن بناتی

Key skills:

- Specific questions □لیے □ کے subagents spawn □ کریں جبکہ high-level coordination main agent میں رکھیں
- Key findings میں بعد اور محفوظ رکھنے refer □ لیے □ کے کرنے scratchpad files استعمال کریں
- □اگلے phase □ کے subagents spawn □ پہلے □س کرنے key findings summarize کریں
- Long investigations دوران □ کے context usage □ لیے □ کے کرنے کم `/compact` استعمال کریں

5.5 Human Oversight اور Confidence Calibration کے ساتھ □ Workflows Design کرنا

Key knowledge:

- Aggregate metrics (97 مثلاً% overall accuracy) specific document types یا fields □ پر کمزوری چھپا سکتے □یں
- Stratified random sampling high-confidence extractions میں error rates measure □□ کرتا

- Field-level confidence calibration labeled validation sets کے ذریعے کریں
- Automation پر سے پہلے document type اور field segment کے حساب سے accuracy validate کریں

Key skills:

- New error patterns detect کرنے کے لیے stratified random sampling implement کریں
- Stable performance validate کرنے کے لیے document type اور field کے حساب سے accuracy analyze کریں
- Field-level confidence scores output کریں اور labeled data کے ذریعے review thresholds calibrate کریں
- Low-confidence یا ambiguous-source extractions human review کی طرف route کریں

5.6 Multi-source Synthesis میں Provenance Preserve کرنا اور Uncertainty Handle کرنا

Key knowledge:

- Summarization کے دوران attribution اگر کھو جاتی ہے “claim → source” mappings محفوظ نہ رہیں
- Aggregation کے دوران structured mappings برقرار رہنی چاہئیں
- Conflicting statistics کو arbitrarily ایک value منتخب کرنے کے بجائے attribution کے ساتھ conflict annotate کر کے handle کریں
- Temporal differences کو contradiction سمجھنے سے بچنے کے لیے publication/collection dates شامل کریں

Key skills:

- Subagents سے “claim → source” mappings (URL، document name، quotes) output کروائیں
- Reports کو stable findings اور disputed ones الگ میں structure کریں
- Conflicting values کو annotations کے ساتھ محفوظ رکھیں اور reconciliation کے لیے coordinator کو دیں
- Correct temporal interpretation کے لیے publication dates شامل کریں
- Content type مطابق کے render کریں: financial data tables میں، news prose میں، technical findings structured lists میں

Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر Claude-based حل بناتے وقت درست trade-off فیصلے کر سکتا ہے یہ امتحان Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) جن بنیادی ٹیکنالوجیز کے بنیادی علم کو جانچتا ہے — یعنی وہ ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی ہیں جن سے Claude

□ لیے: کسٹمر سپورٹ کے □ سوالات حقیقی دنیا کے □ منظرناموں پر مبنی □ ہوتے □ ہیں امتحان کے □ agentic systems بنانا، developer productivity میں شامل کرنا CI/CD کو Claude Code ڈیزائن کرنا، multi-agent research pipelines بنانا، اور غیر ساختہ □ دستاویزات سے □ tools نکالنا□ structured data

---

□دف امیدوار

مثالی امیدوار ایک **solution architect** جو □□ Claude اور ڈیزائن ایپلیکیشنز پروڈکشن ساتھ کے □ ship کرتا □□□ آپ کہ پاس کم از کم 6 ماہ □ کا عملی تجربہ □ ہونا چاہیے□:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration □ لیے □ کے tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

---

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 درست 1 □ سے میں)
اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	(□ر سوال کا جواب دیں!) نہیں
Scenarios	8 4 □ سے میں (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent جو بناتے ہیں Claude Agent SDK استعمال کرتے ہوئے returns، billing disputes، اور account issues کو سنبھالتا ہے agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) 80 فیصد+ first-contact resolution مناسب escalation کے ساتھ استعمال کرتا ہدف ہے۔

Scenario 2: Claude Code ساتھ کے Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں کہ تیز کرنے: code generation، refactoring، debugging، documentation۔ آپ کو اس کے ساتھ CLAUDE.md configuration اور custom slash commands کے استعمال کو سمجھنا ہے کہ planning mode کرنا ہے، اور integrate

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks سونپتا ہے: web research، document analysis، synthesis، اور report generation۔ سسٹم کو citations کے ساتھ مکمل رپورٹس تیار کرنی چاہئیں

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، اور MCP servers اور Built-in tools (Read، Write، Bash، Grep، Glob) استعمال کرنے میں مدد دیتا ہے automate ہوتے ہیں۔

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback ہو۔ حاصل Prompts کہ چاہئیں بننے ایسے false positives کم سے کم ہوں۔

Scenario 6: Structured Data Extraction

ہے معلومات نکالتا سے دستاویزات ساختہ غیر سسٹم، output کو JSON schemas کے ذریعے کہ validate کرتا ہے، اور high accuracy اسے ہے برقرار رکھتا edge cases کو چاہئیں درست طور پر سنبھالنے

Scenario 7: Conversational AI Architecture Patterns

آپ multi-turn conversational systems میں جن ڈیزائن کرتے ہیں context window management، متعدد turns میں instructions کا برقرار رہنا، memory strategies، محفوظ execution کے لیے tool design، اور مبہم یا متضاد user inputs کو سنبھالنا شامل ہے

Scenario 8: Agentic AI Tools *(ہماری مدد کریں — ہیں نہ مواد موجود!)*

اس scenario والے کی اطلاع امتحان دینے candidates اس تک ابھی ہے لیکن یہ ہے دی ہے نہ guide میں شامل نہیں GitHub Issues میں share کریں کہ سوالات دیکھے ہیں، تو انہیں scenario ہے اگر آپ نے اصل امتحان میں اس شامل کر سکیں۔ آپ کا تعاون ہر اس شخص کی مدد کرے گا جو امتحان کی تیاری کر coverage تاکہ ہم مکمل رہا ہے

□حصہ I: نظریاتی بنیادیں

یہ حصہ وہ بنیادی نظریہ بیان کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے مواد کو topic کے مطابق — اس سے ہٹ کر domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بنتی ہے

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude Messages API request میں عموماً یہ ایک request–response model پر کام کرتا Claude API شامل ہوتا ہے:

```
{
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "system": "You are a helpful assistant.",
    "messages": [
        {"role": "user", "content": "Hi!"},
        {"role": "assistant", "content": "Hello!"},
        {"role": "user", "content": "How are you?"}
    ],
    "tools": [...],
    "tool_choice": {"type": "auto"}
}
```

اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens
- `system` — system prompt (model کی تعریف کرنے والی)
- `messages` — conversation history (**full history بھیجنا ضروری ہے**)
- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array میں تین roles ہیں جو ہوتے ہیں:

- `user` — user messages
- `assistant` — model responses (history میں شامل ہوتے وقت بھیجتے)

- `tool` — tool call results (عملاً `tool_result` content block)

اہم بات: ہر API request میں مکمل conversation history بھیجی Model requests درمیان کی state محفوظ آزاد ہوتی ہیں رکھتا؛ ہر call

1.3 `stop_reason`

Claude API response میں `stop_reason` کی جو بتاتا ہے کہ model کیوں رُکا ہوتا ہے:

Value	Description	Action
`"end_turn"`	Model نے جواب مکمل کر لیا	user کو نتیجہ دکھائیں
`"tool_use"`	Model tool call چاہتا ہے	tool چلائیں اور result واپس دیں
`"max_tokens"`	Token limit ختم ہو گئی	response truncated؛ limit بڑھا سکتے ہیں
`"stop_sequence"`	Stop sequence مل گئی	application logic کے مطابق handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` اہم سب سے signals یہی — ہیں loop control کرتے ہیں۔

1.4 System Prompt

System prompt ایک خاص instruction جو ہے context اور behavioral rules یہ ہیں متعین کرتا ہے:

- `messages` array الگ؛ ہوتا نہیں حصہ کا `system` field ہے میں دیا جاتا
- user messages پر فوقیت رکھتا ہے
- ایک بار load ہو کر پوری گفتگو میں لاگو رہتا ہے
- role، constraints، اور output format ہے ہوتا استعمال کی وضاحت کے لیے

امتحان کے لیے اہم: system prompt کی wording tool selection مثال کو غیر ارادی طور پر متاثر کر سکتی ہیں ضرورت سے زیادہ `get_customer` کو model کی تصدیق کریں" جیسی ہدایت کے طور پر "ہمیشہ استعمال کرنے پر لے جا سکتی ہیں

1.5 Context Window

Context window کل متن وہ (tokens) جسے ہے model ایک وقت میں process اس میں شامل کر سکتا ہیں ہے:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

اہم مسائل:

1. **Lost-in-the-middle effect:** لمبے input کی درمیان، ہیں ہوتی مؤثر زیادہ کے آغاز اور اختتام کی معلومات ہو سکتی ہیں۔ حل: اہم معلومات شروع یا آخر میں رکھیں details miss

2. **Tool results کا جمع کرنا:** ہر tool call context میں output اگر 40+ fields واپس کرتی شامل کرتی ہے تو 5 اہم لوں مگر صرف context کرتا ضائع ہوتا ہے
3. **Progressive summarization:** history compress پر کرنے numeric values، percentages، اور dates اکثر vague ہو جاتی ہیں

---

2 باب: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` کیا ہے

`tool_use` وہ mechanism ہے ذریعے جس کے Claude external functions call کرتا ہے Model براہ کر سکتا واپس کرتا result کر کے execute اسے code بناتا ہے؛ آپ کا structured tool call نہیں چلاتا — وہ code راست ہے

2.2 Tool Definition

ہر tool JSON schema کے ذریعے define ہوتا ہے:

```
{
    "name": "get_customer",
    "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
    "input_schema": {
        "type": "object",
        "properties": {
            "email": {"type": "string", "description": "Customer email"},
            "customer_id": {"type": "integer", "description": "Numeric customer ID"}
        },
        "required": []
    }
}
```

Tool description کے اہم نکات:

1. Description ہے tool selection کا بنیادی mechanism ہے Minimal descriptions غلط selection کا سبب بنتی ہیں
2. Description میں یہ شامل کریں: tool کیا ہے، کیا کرتا ہے، input format کیا ہے، edge cases کیا ہیں، اور یہ کب استعمال ہو
3. Overlapping descriptions سے بچیں
4. Built-in tools اور MCP tools میں overlap ہو تو MCP tool کو بہتر اور واضح بنائیں

2.3 `tool_choice`

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model □□ کرتا خود ط□	عام default
`{ "type": "any" }`	Model کو لازماً tool call کرنی ہوگی□	جب structured output چا□یے□
`{ "type": "tool", "name": "extract_metadata" }`	خاص tool لازماً call ہوگی□	forced first step / order ک□ لیے□

2.4 JSON Schemas for Structured Output

JSON schemas ساتھ ک□ `tool_use` استعمال کرنا Claude □سے structured output □لینے کا سب سے reliable طریقہ □□□ ہے□ syntax errors اور □□ کم کرتا required structure، مگر □□ نافذ کرتا semantic correctness کی ضمانت نہ□یں دیتا□

Design اصول:

1. صرف وہ fields required رکھیں جو □میشہ ملنے چا□ئیں□
2. □ممکنہ missing data لیے ک□ nullable fields استعمال کریں□
3. `"other"` اور `"unclear"` جیسے□ enum values شامل کریں تاک□ information □ہو□ ضائع نہ□

2.5 Syntax اور Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema ساتھ ک□ `tool_use`
Semantic	Totals match □کرتے نہ□یں، value غلط field میں	Validation checks، feedback retry

3 باب: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop

Agentic loop میں Claude □بلکہ، صرف جواب نہ□یں دیتا actions کی sequence □□ چلاتا:

1. Claude کو tools ساتھ ک□ request بھیجیں
2. Response لیں
3. `stop_reason` چیک کریں:
   - `"tool_use"` → tool چلائیں، result history □پھر دوبار□ میں ڈالیں، request

- `"end_turn"` → task مکمل، نتیجہ user کو دیں
4. مکمل ہونے تک دہرائیں

درست signal: completion کے لیے `stop_reason == "end_turn"` دیکھیں۔ Text parsing یا arbitrary iteration limit قابلِ اعتماد نہیں ہیں۔

3.2 `AgentDefinition`

```
agent = AgentDefinition(
        name="customer_support",
        description="Handles customer requests for returns and order issues",
        system_prompt="You are a customer support agent...",
        allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
)
```

اہم چیزیں: `name`، `description`، `system_prompt`، اور `allowed_tools`۔ Least privilege اصول اپنائیں۔

3.3 Coordinator اور Subagents

Multi-agent systems اکثر hub-and-spoke topology استعمال کرتی ہیں:

```
                Coordinator
               /     |      \
    Subagent1  Subagent2  Subagent3
```

Coordinator task کو توڑتا ہے، subagents کو منتخب کرتا ہے، کام سونپتا ہے، نتائج جوڑتا ہے، errors handle کرتا ہے، اور user تک final جواب پہنچاتا ہے۔

اہم اصول: subagents کا context خود بخود inherit نہیں کرتے؛ وہ الگ ہوتا ہے parent history

3.4 `Task` Tool

Subagents `Task` tool کے ذریعے spawn کیے جاتے ہیں۔ Coordinator کے `allowed_tools` میں `"Task"` ہونا چاہیے۔

صحیح context passing:

- `Task: "Analyze the document"` کافی نہیں
- `Task: "Analyze this document. Full text: ... Output schema: ..."` درست ہے

Coordinator ایک response میں multiple `Task` calls بھیج سکتا ہے، لہٰذا parallel subagents ممکن ہیں۔

3.5 Hooks

Hooks lifecycle کے مخصوص points پر work کرتے ہیں۔

PostToolUse tool result کو model کو پہنچانے سے پہلے normalize کر سکتا ہے:

```
@hook("PostToolUse")
def normalize_dates(tool_result):
        return normalized_result
```

PreToolUse policy violation block کر سکتا ہے:

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
        if tool_call.name == "process_refund" and tool_call.args.amount > 500:
                return redirect_to_escalation(tool_call)
```

یاد رکھیں: hooks deterministic ہیں؛ prompt instructions probabilistic ہیں۔ Critical business rules کے لیے hooks بہتر ہیں۔

باب 4: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP کیا ہے

MCP ایک open protocol ہے جو external systems کو Claude کے تین بنیادی طریقوں سے جوڑتا ہے۔ اس کے تین resource types ہیں:

1. **Tools** — actions کے لیے functions
2. **Resources** — context data (documentation، schemas، catalogs)
3. **Prompts** — predefined task templates

4.2 MCP Servers

MCP server ایک process ہے جو tools/resources expose کرتا ہے۔ Connect ہونے پر tools discover ہو جاتے ہیں اور descriptions کے مطابق استعمال ہوتے ہیں۔

4.3 Configuration

Project config (`.mcp.json`) کے لیے ٹیم کے:

```
{
    "mcpServers": {
        "github": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-github"],
            "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
        }
    }
}
```

Project config version control میں ہوتی ہیں؛ secrets environment variables سے آتی ہیں User config (`~/.claude.json`) personal/experimental servers کے لیے ہوتی ہیں

4.4 `isError`

MCP tool errors میں `isError: true` استعمال کریں Generic errors بجائے structured error واپس کریں:

```
{
    "isError": true,
    "content": {
        "errorCategory": "transient",
        "isRetryable": true,
        "message": "The service is temporarily unavailable."
    }
}
```

4.5 Resources

Resources وہ data ہیں جو agent بغیر action کے پڑھ سکتا ہے: schemas، docs، content catalogs، summaries یا exploratory tool calls میں کمی کرتے ہیں

باب 5: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

CLAUDE.md تین سطحوں پر ہو سکتا ہے:

- **User-level:** `~/.claude/CLAUDE.md` — صرف اسی user پر لاگو
- **Project-level:** `.claude/CLAUDE.md` یا root `CLAUDE.md` — پوری team کے لیے
- **Directory-level:** subdirectories میں CLAUDE.md — مخصوص folder conventions

5.2 `@path` Syntax

CLAUDE.md میں `@path` سے external files refer کیے جا سکتے ہیں تاکہ modular config:

```
Coding standards @./standards/coding-style.md میں
Test rules @./standards/testing-requirements.md میں
```

5.3 `.claude/rules/`

`.claude/rules/` rules کو topic کے لحاظ سے organize کرتا ہے اور `paths` کے ذریعے conditional loading دیتا ہے:

```
---
paths: ["src/api/**/*"]
---

API files کے لیے async/await اور explicit error handling استعمال کریں
```

5.4 Commands اور Skills

`~/.claude/commands/` کے لیے ہے، جبکہ project commands `.claude/commands/` میں frontmatter کے ساتھ skills ملتی ہیں

5.5 Skills اور `context: fork`

`context: fork` skill کو isolated context میں چلاتا ہے `allowed-tools` سے tool access محدود کریں، اور `argument-hint` سے required input مانگیں

5.6 Planning Mode

Planning mode بڑی architectural تبدیلیوں، multiple approaches، اور unfamiliar codebases کے لیے بہترین ہے Direct execution چھوٹے اور واضح changes کے لیے بہتر ہے

5.7 `/compact` اور `/memory`

`/compact` context compress کرتا ہے، اور `/memory` preferences اور notes سنبھالنے میں مدد دیتا ہے

5.8 Claude Code CLI for CI/CD

`-p` / `--print` non-interactive mode □□ لیے □وتا استعمال کا □:

```
claude -p "Analyze this pull request for security issues"
```

Structured output □لیے کا □ `--output-format json` اور `--json-schema` استعمال کریں□

---

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting 4–2 میں examples کر □د expected behavior □یہ □□□ دکھایا جاتا vague instructions □سے کیونکہ □□ مؤثر زیاد□ example ambiguity □□□ کم کرتا

6.2 Explicit Criteria

Vague instructions □بجائ□ کہ clear criteria کیا :دیں flag کیا، □□ کرنا ignore اور، □□ کرنا severity □کیسے decide □□□ کرنی

6.3 Prompt Chaining

Complex task کو focused steps میں توڑیں: local analysis، پھر integration pass□یہ □ attention dilution کم کرتا □□□

6.4 Interview Pattern

Claude □سے clarifying questions پوچھوانا design ambiguity جب خاص طور پر، □□ لاتا □سامنے domain unfamiliar □و□

6.5 Validation and Retry

Extraction fail تو □و original document، incorrect output، اور specific validation error ساتھ □کا retry □کریں

6.6 Self-correction

Model stated value اور calculated value دونوں نکالو؛ conflict ہو تو discrepancy handle کریں۔

باب 7: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API asynchronous processing 50 کم% savings، up to 24 hours window، اور multi-turn tool calling supported ہیں۔ `custom_id` سے requests اور responses link ہوتے ہیں۔

7.2 کب استعمال کریں

Pre-merge checks اور interactive tasks کے لیے synchronous API، جبکہ overnight reports اور bulk jobs کے لیے Batch API استعمال کریں۔

7.3 `custom_id`

`custom_id` result کو original document سے جوڑنے، failures identify کرنے، اور صرف failed items دوبارہ submit کرنے میں مدد دیتا ہے۔

باب 8: Task Decomposition Strategies

8.1 Fixed Pipelines

Predictable tasks کے لیے fixed pipeline جیسے Document → Extraction → Validation → Final output استعمال کریں۔

8.2 Dynamic Decomposition

Open-ended tasks میں subtasks intermediate results کی بنیاد پر بنتے ہیں۔

8.3 Multi-pass Code Review

Large PRs کے لیے per-file analysis کے بعد cross-file integration pass کریں تاکہ attention dilution کم ہو۔

باب 9: Escalation اور Human-in-the-Loop

9.1 کب Escalate کریں

Explicit user request، policy gaps، progress ہونا □□، threshold اوپر □□ financial action، یا ambiguous matching کی صورت میں escalate □کریں Sentiment analysis اور self-rated confidence unreliable □ہیں

9.2 Escalation Patterns

□□ مسئلہ □□پ resolve اگر پھر، کی کوشش کریں □کرنا policy gap یا explicit insistence □و تو human کو hand off کریں□

9.3 Structured Handoff

Escalation پر customer ID، issue summary، actions taken، اور recommended action □شامل کریں تاکہ human operator کو مکمل context □□ملے

9.4 Confidence Calibration

Field-level confidence scores، calibration on labeled data، اور stratified sampling □استعمال کریں

باب 10: Multi-agent Systems میں Error Handling

10.1 Error Categories

Transient errors retryable □ہیں، validation errors input fix □ہیں مانگتی، business errors policy-driven □ہوتی ہیں، اور permission errors generally escalate □ہیں□ کی جاتی

10.2 Anti-patterns

Generic “search unavailable” response، silent suppression، یا پورا workflow abort □□□ □کرنا نقصان دہ Structured errors □واپس کریں

10.3 Structured Subagent Error

Structured error میں failure type، attempted query، partial results، alternative approaches، اور coverage impact □شامل کریں

10.4 Final Synthesis میں Coverage

Report میں واضح کریں کہ کون سی sections fully covered ہیں، کون سی partially covered ہیں، اور کہاں gaps باقی ہیں

باب 11: Production Systems میں Context Management

11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block میں رکھیں تاکہ summarization باوجود کبھی critical data ضائع نہ ہو

11.2 Tool Results Trimming

PostToolUse hook صرف relevant fields رکھیں

11.3 Position-aware Input

Important findings شروع میں اور action items آخر میں رکھیں تاکہ lost-in-the-middle effect کم ہو

11.4 Scratchpad Files

Verbose findings scratchpad میں لکھیں تاکہ long investigations میں context محفوظ رہے

11.5 Subagents کو Delegate کرنا

Exploration subagents میں دیں اور main context میں صرف summary رکھیں

11.6 Structured State Persistence

ہر agent اپنی state manifest یا JSON file میں export کرے تاکہ crash recovery ممکن ہو

باب 12: Provenance کو محفوظ رکھنا

12.1 Attribution Loss

Claim کے ساتھ source URL، publication date، اور confidence رکھیں

12.2 Conflicting Data

اگر sources مختلف values دیں تو دونوں کو attribution ساتھ محفوظ رکھیں اور conflict flag کریں

12.3 Dates

Temporal differences کو contradiction سمجھنے سے پہلے publication dates دیکھیں

12.4 Content Type مطابق کے Render

Financial data tables میں، news prose میں، technical findings structured lists اور میں time series chronological order میں دکھائیں

---

13 باب: Claude Code کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
pattern سے file تلاش کرنا	**Glob**	`**/*.test.tsx`
file contents میں search	**Grep**	function name, error message
file پڑھنا	**Read**	analysis کے لیے
نیا file بنانا	**Write**	scratch سے
precise edit	**Edit**	unique text match
shell command	**Bash**	git، npm، tests

13.2 Incremental Investigation

ایک ساتھ سب files پڑھیں؛ نہ entry points، پھر usages، پھر consumer files دیکھیں

13.3 Read + Write Fallback

اگر Edit fail ہو تو Read کریں، تبدیلی programmatically کریں، پھر Write کریں

حصہ II: امتحانی ڈومین نوٹس

ڈومین 1: Agent Architecture اور Orchestration (27%)

Agentic loops، coordinator–subagent architecture، `Task` ٹول کے ذریعے spawning، hooks، escalation، اور multi-step workflows پر فوکس کریں۔

ڈومین 2: Tool Design اور MCP Integration (18%)

Tool descriptions، `tool_choice`، JSON schema output، structured errors، MCP servers، resources، اور built-in tools کے فرق کو سمجھیں۔

ڈومین 3: Claude Code Configuration اور Workflows (20%)

CLAUDE.md hierarchy، `.claude/rules/`، commands، skills، planning mode، `context: fork`، اور CI/CD integration کو عملی طور پر سمجھیں۔

ڈومین 4: Prompt Engineering اور Structured Output (20%)

Few-shot prompting، explicit criteria، prompt chaining، validation/retry loops، JSON schema enforcement، اور batch-friendly prompting پر توجہ دیں۔

ڈومین 5: Context Management اور Reliability (15%)

Summarization risks، lost-in-the-middle، trimming tool output، confidence calibration، escalation، اور provenance preservation کو یاد رکھیں۔

Scenario 7 کے مشق سوالات: Conversational AI Architecture Patterns

سوال 61

صورتحال: آپ کے `remove_team_member` tool میں execution کے اثرات کا پتہ لگانے سے preview کرنے کے لیے ایک `dry_run: boolean` parameter شامل کر رہے ہیں۔ Production monitoring سے معلوم ہوتا ہے کہ agent براہ راست `dry_run=false` کے ساتھ call کر کے preview step کو bypass کر دیتا ہے، یقینی بنانا ہے کہ آپ کو یہ explicitly confirm کرنا ہوگا کہ user کو جس preview سے پتہ لگا ایک deletion کیا ہوگا۔

کونسی approach سب سے قابل اعتماد ہے؟

- A) Server-side validation شامل کریں جو `dry_run=false` صرف اسی وقت allow 60 کرے جب پچھلا `dry_run=true` call ہوئی ہو parameters کے ساتھ میں یکساں seconds
- B) Tool کو confirmation کی ضرورت کے طور پر annotate کریں اور orchestration layer کو configure کریں کہ user کی approval مانگے سے پہلے calls forward کرنے کو annotated tools

- C) Tool description میں تفصیلی instructions اور few-shot examples شامل کریں جن میں agent کو مطالبہ کرنا چاہیے کہ وہ `dry_run=true` کے ساتھ call کرے، نتیجہ پڑھے، اور پھر actual call کرے
- D) دو tools کے بجائے replace کریں: `preview_remove_member` ایک impact details اور single-use confirmation token واپس کرتا ہے؛ `execute_remove_member` کو وہ token درکار ہے جو صرف اسی preview کے execution کو bind کرتا ہے **[درست]**

D کیوں: Token-binding approach یہ architecturally پکا کرتا ہے کہ بغیر preview کے execution ناممکن ہو جاتا ہے — execute tool کو literally وہ token جو صرف preview tool ہی generate کر سکتا ہے، چاہے LLM instructions کی پابندی (C)، timing heuristics (A)، یا orchestration infrastructure (B) جو بھی approach واحد ہے جو code level پر constraint enforce کرتی ہے، انحصار کرنے کے بجائے

سوال 62

صورتحال: Production monitoring سے معلوم ہوتا ہے کہ آپ کا `search_catalog` tool 12% وقت fail ہوتا ہے: 8% network timeouts جو آپ retry 4 بار کامیاب ہوتے ہیں، اور 4% query syntax errors جو کبھی کامیاب نہیں ہوتے۔ دونوں فی الحال یکساں طریقے سے return ہونے والے error types ہیں، جس سے بچنا فائدہ مند ہوتا ہے

آپ tool کی error handling کیسے modify کریں گے؟

- A) System prompt میں few-shot examples شامل کریں جو demonstrate کریں کہ network errors اور syntax errors میں فرق کیسے کریں
- B) تمام errors پر uniformly exponential backoff retry logic apply کریں
- C) Tool کے اندر network timeouts کے لیے automatic retry with backoff implement کریں؛ syntax errors کو parameter validation details کے ساتھ فوری واپس کریں **[درست]**
- D) تمام errors کو `retryable` boolean flag اور error type details کے ساتھ واپس کریں

C کیوں: Tool level پر transient errors کے لیے retries handle کرنا صحیح abstraction boundary ہے — tool کو agent پر deterministic retry logic implement کرنے بغیر کہ بار بار میں یقینی علم ہو اور وہ error type کے انحصار کے کہ وہ flag (D) interpret یا prompt instructions (A) follow کرے Uniform backoff (B) syntax errors پر وقت ضائع کرتا ہے جو کبھی کامیاب نہیں ہوں گے

سوال 63

صورتحال: Investment strategy کی کئی turns میں، user کی "میری risk tolerance اور بہت کم ہے" نہ کہنا "کرنا چاہیے؟ invest کرنا چاہتا ہوں" اب وہ پوچھتا ہے: "مجھے کس چیز میں returns maximize پھر "میں اپنے

کونسا approach کے کہ recommendation user کی اصل priority سے سب سے بہتر طریقے سے یقینی بناتا ہے آلنگ ہو؟

- A) سے پوچھیں کہ کونسی چیز زیادہ اہم ہے **[درست]** user تضاد کو سامنے لائیں اور
- B) دونوں scenarios کے لیے الگ الگ recommendations فراہم کریں
- C) سب سے حال میں بیان کی گئی preference کے ساتھ آگے بڑھیں
- D) تضاد کو حل کیے بغیر balanced portfolio تجویز کریں

کیوں: جب A کی preferences اور user براہ راست متضاد ہوں، تو تضاد سامنے لانا اور clarification مانگنا ہی واحد طریقہ ہے کہ recommendation user کا اصل ارادہ سے ہم آہنگ ہو Returns maximize کرنا اور low risk بنیادی طور پر ناقابل مطابقت اہداف ہیں جن کے لیے انسانی فیصلہ درکار ہے tolerance

---

سوال 64

صورتحال: Users کئی conversation turns میں playlist preferences refine کرتے ہیں User مجھے "jazz کے بعد، Claude سے messages پسند ہیں؟" پوچھتا ہے "آپ کو کون سے genres پسند ہیں" کہنے کے دو

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) Claude کو conversation memory برقرار رکھنے کے لیے vector database connection درکار ہے
- B) Model کی context window exceed ہو گئی ہے
- C) Claude API کو `session_id` parameter درکار ہے
- D) آپ کی application `messages` array میں پچھلے messages **[درست]** شامل نہیں کر رہی

کیوں D: Claude پاس کے server-side memory ہر API call stateless نہیں ہے — ہر request کے ساتھ `messages` array میں مکمل conversation history بغیر شامل کیے، Claude کے پاس پچھلے turns کا کوئی علم نہیں ہوتا Vector databases (A) اور `session_id` (C) Claude کے architecture کا حصہ نہیں ہیں؛ دو ناممکن ہے (B) context window overflow کے لیے کے exchange messages

---

سوال 65

صورتحال: 40 منٹ کے cooking session بعد کے، conversation 78,000 tokens تک پہنچ گئی ہے History میں شامل ہیں آپ کو اہم معلومات discussion اور عمومی، clarified cooking terms، recipe scaling، allergies محفوظ رکھنے ہوئے tokens کم کرنے ہیں

کونسا approach preservation اور token reduction کے درمیان بہترین توازن رکھتا ہے؟

- A) پوری conversation history کا خلاصہ کریں
- B) 20,000 صرف سب سے حالیہ tokens رکھیں
- C) اہم structured data (allergies، quantities، preferences) عمومی نکالیں، اور discussion کا خلاصہ کریں، حالیہ exchanges کو verbatim **[درست]** رکھیں
- D) مکمل conversation externally store کریں اور semantic search حصے کے ذریعے متعلقہ retrieve کریں

کیوں C: Hybrid approach کم سے کم cost پر سب سے زیادہ valuable معلومات محفوظ رکھتی ہے Allergies اور recipe quantities جیسے critical facts ایک compact structured block میں extract ہوتے ہیں (summarization exchanges ہوتی ہے، اور حالیہ، عمومی discussion summarize (سے بچاتے ہوئے precision loss کے دوران conversational coherence لیے کے verbatim رہتے ہیں Options A اور B اہم dietary information کھو سکتے ہیں؛ D ایک cooking session ہے کے لیے ضرورت سے زیادہ پیچیدہ

سوال 66

صورتحال: Users طویل □یں کئ □ conversations میں assistant □ک □ہلا□ پ topics اور preferences کا رپورٹ کرت□ □یں۔ message pairs □□□ رکھتی صرف آخری 25 implementation حساب کھو دیتا □□□ آپ کی موجود□

کیا □□؟ solution سب سے مؤثر

- A) Hybrid approach: پرانے messages □ حالیہ □وئ□ کرت□ خلاص□ کا messages verbatim **[درست]** رکھیں□
- B) مکمل conversation history پر vector similarity search□
- C) Window 50 کو message pairs □ تک بڑھائیں
- D) ہر turn میں dropped messages اور running summary کا خلاص□ کریں □ آگ□ جوڑیں

کیوں A: Hybrid approach □□ دونوں پ□لوؤں کو حل کرتی □□ کئ مسئل□: exact recent context □□ محفوظ رکھتی (conversational coherence □م ا□ لی□ کئ) کی compressed representation برقرار جبک□ preferences کی پ□لا□ □□□ رکھتی Window بڑھانا (C) □□□ ملتوی کرتا اسی مسئل□ کو Vector search (B) □م ا context کو miss مشاب□ نہ □و□ semantically سے current query کر سکتی □□ جو

سوال 67

صورتحال: Users جب □یں کئ رپورٹ کرت□ □یں conversations 50 turns تو □وت□ □یں latency بڑھ جاتی سے زیاد□ اور □□ costs بھی□

بنیادی وج□ کیا □□؟

- A) ہر API request ساتھ پوری کئ conversation history **[درست]** □□□ شامل □وتی
- B) Model gradually لمبے responses generate □□□ کرتا
- C) History ساتھ کئ بڑھن□ database operations □یں□ سست □و جات□
- D) Model زیاد□ processing والا ضرورت کی internal user profile □□□ بناتا

کیوں A: Claude کی API مکمل طور پر stateless □ہر — □□ request کو `messages` array میں پوری □یں، ہر request بڑھتی زیاد□ conversations شامل کرنی □وتی □□□ جیس□ جیس□ conversation history دونوں بڑھاتا □□□ cost اور directly processing latency □ جاتی □□، جو tokens کئ درمیان کوئی Model calls internal state رکھتا نہیں (D □□ غلط)□

سوال 68

صورتحال: تین م□ینوں کئ weekly sessions بعد کئ، conversation history 85,000 tokens جب تک بڑھ گئی □□□ پچھلی discussions assistant کئ بار□ میں کیا نتیج□ اخذ کیا تھا؟"، تو theme کئ isolation پوچھتا □□ "□م نہ user □ reference □□□ کرن□ کئ بجائ□ عام جوابات دیتا

کیا □□؟ approach سب سے مؤثر

- A) Rolling window truncation□
- B) Key conclusions capture □وئ□ کرت□ progressive summarization□
- C) Relevant exchanges retrieve □ لی□ کئ کرن□ semantic embeddings**[درست]** □
- D) Discussion conclusions mark □ لی□ کئ کرن□ structured XML tags شامل کریں□

C کیوں: Conversation history پر semantic search واحد approach کی جو تین مہینوں کو discussion scale کر سکتی ہے جبکہ demand پر specific relevant exchanges surface کر سکتی Rolling window truncation (A) history کا بیشتر حصہ ضائع کر دے گی Progressive summarization (B) discussions کو abstractions میں compress کرتی ہے، specific conclusions جن کی بار بار میں ضرورت ہوتی ہے users کھو دیتی ہیں پوچھتے

سوال 69

صورتحال: QA testing دوران کہ، Claude 15–10 پہلے turns میں system prompt guidelines follow کرتا ہے، Conversation ابھی token limits کے اندر آ جاتی ہے deviation میں responses لیکن بعد کے

کیا ہے؟ بہترین solution

- A) Behavioral guidelines کو پہلے user message میں منتقل کریں
- B) 20 turns بعد نئی کہ conversation شروع کریں
- C) Conversation breakpoints پر guidelines reinforce کرنے والے user-role messages insert **[درست]** کریں
- D) Non-compliant responses کو regenerate کرنے کے لیے post-response validation استعمال کریں

C کیوں: Behavioral reminders کا periodic injection instruction drift کرتا ہے راست مقابلہ کا، history کو پہلے Guidelines کر کے user regular intervals پر constraints re-establish ہونے کے ساتھ accumulate message میں منتقل کرنا (A) ان کا authority کم کرتا Post-response validation (D) corrective ہے نہ کہ preventive اور significant latency شامل کرتی

سوال 70

صورتحال: آپ کے AI tutor 2,800-token system prompt جو teaching methodology اور adaptation rules کو نظرانداز کرنا شروع کر دیتا ہے، assistant proficiency levels کے بعد turns کرتا ہے define 12

کیا ہے؟ fix سب سے مؤثر

- A) 5–4 ہر turns میں reminders inject کریں
- B) Verbose rules کو proficiency-level adaptation demonstrate کرنے والے few-shot examples سے replace کریں **[درست]**
- C) Critical rules کو system prompt کے آخر میں رکھیں
- D) Responses evaluate کریں اور اگر difficulty level match نہ ہو تو regenerate کریں

B کیوں: Declarative rules 2,800-token system prompt drift کے vulnerable ہیں کیونکہ abstract rules کو concrete few-shot examples سے replace کرنا جو correct proficiency-level adaptation demonstrate کریں، model کو match کرنے کے لیے clear behavioral patterns کئی ہیں — یہ دیتا turns میں abstract instructions سے زیادہ reliably follow ہوتا ہے

سوال 71

صورتحال: آپ کے assistant کو enthusiastic tone برقرار رکھنا، اپنی reasoning explain کرنا، اور clarifying questions پوچھنے ہیں۔ یہ behavioral guidelines کہاں define ہونی چاہئیں؟

□ی behavioral guidelines □ہاں □کیسے define □ونی چاہ□ئیں؟

- A) □ر user message □ہلے □سے prepend □کریں
- B) System prompt **[درست]** □میں
- C) □ہلے assistant message □میں
- D) Environment variables □میں

B کیوں: System prompt □ہ□ persistent behavioral □ونے□ والی apply میں conversation خاص طور پر پوری □ہلے □سے (A) redundant □ہ□ شامل کرنا user message □کو لیے □□□ ڈیزائن کیا گیا □ہ□ guidelines اور constraints □□□ assistant message (C) unreliable □کیونکہ □□ model □ہلے □اپنے statements □سے deviate □□□ کر سکتا Environment variables (D) model □کے behavior □□ڈالتے □ہیں اثر پر

سوال 72

صورتحال: Users repetitive response openings □جیسے □"یقیناً!" اور "میں مدد کرنے□ میں خوش □وں!" رپورٹ کرتے□ □ہیں

کیا □□؟ approach سب سے □مؤثر

- A) Direct response opening ساتھ □کے partial assistant message append **[درست]** □کریں
- B) Temperature setting □کم کریں
- C) Greetings □لیے □کے □ٹانے□ responses کو post-process □کریں
- D) System prompt میں ان phrases کی □بچنے□ سے instructions □شامل کریں

A کیوں: Assistant □کے response کو direct □سے جواب کی شروعات prefill کرنا generation level پر greeting patterns □□ روکتا — model prefill □سے continue □نئے□ بجائے□ □□ کرتا opening phrases generate □□کرنے□ System prompt instructions (D) مدد کر سکتی □ہیں لیکن کم reliable □ہیں□ Post-processing (C) ایک fragile workaround □□□ Temperature (B) randomness control □□کرتا، specific phrase patterns □نہیں

سوال 73

صورتحال: ایک webhook □کہ آپ کے system کو notify □کہ □□ کرتا user کا package ship □جبکہ □□ ہو گیا□ user actively chat □کہ □ہیں آپ چاہتے□ □□□ کر رہا assistant □سے اگلا response میں naturally □□شامل کرے

کیا □□؟ approach بہ□ترین

- A) Shipping status کو system prompt □میں شامل کریں
- B) فوری synthetic user message □بھیجیں
- C) □ر turn میں assistant کو status tool call □کرنے□ پر مجبور کریں
- D) Status update □کو اگلے□ user message □کے prefix □کے طور پر append **[درست]** □کریں

D کیوں: Status update □کو اگلے□ user message □کے prefix □کے طور پر شامل کرنا flow کو interrupt بغیر □کیے natural conversation boundary پر real-time context inject □□□ کرتا System prompt modify کرنا (A) session rebuild □□□ کی ضرورت Synthetic user message (B) natural dialogue flow کو disrupt □ر □□□ کر سکتا turn میں tool call force کرنا (C) اس وقت wasteful جب □□ events rare □ہوں□

سوال 74

صورتحال: Users □ لیے کے "پارٹی اکثر venue book □جیسے" کریں requests □میں □ بھیجتے Assistant 4+ clarifying questions 35 □ جس سے، پوچھتا □□ abandonment % □□□ ہوتا□

کونسا approach trade-off سے □بہترین طریقے کو improve کرتا □□؟

- A) Hidden defaults □کے ساتھ آگے بڑھیں
- B) تمام clarifying questions ایک compound message □میں پوچھیں
- C) Assumptions explicitly بیان کریں اور corrections **[درست]** □مدعو کرتے □ہوئے آگے بڑھیں
- D) Structured intake form □استعمال کریں

C کیوں: Assumptions explicitly بیان کرنا user کو فوری، مفید response غلط □جبکہ دیتا □□ آگے بڑھنا اور Hidden defaults (A) user کو نہیں بتاتے کہ کیا کرنے کی صلاحیت برقرار رکھتا □□□ assumptions کو correct کیا گیا assume Compound question list (B) پھر بھی user سے upfront effort □□□ کا تقاضا کرتی Structured form (D) کم friction □□□ کی بجائے زیادہ شامل کرتا □□□

سوال 75

صورتحال: آپ کا assistant contractor-persona system prompt ابتدائی □□□ استعمال کرتا turns rules follow □□□ دیتا assistant generic advice □□□ تک turn 7 کرتے □ہیں، لیکن Conversation length 2,500 صرف tokens

سب سے زیادہ ممکنہ وجہ کیا □□؟

- A) System prompts صرف initial behavior establish □کرتے □ہیں
- B) Turns accumulate ساتھ کے □ہونے model کی attention □□□ کمزور □ہوتی
- C) Accumulated assistant responses system prompt کو اثر کے dilute **[درست]** □کرتے □ہیں
- D) System prompt □□□ صرف ایک بار بھیجا جاتا

C کیوں: جیسے جیسے conversation history میں assistant responses □ہوتے □ہیں جمع، system prompt کے assistant-generated content کے بڑھتے □ہوئے proportion کا text کرنے والے reflect کو behavioral constraints کے بجائے اپنے پہلے system prompt instructions پر بھی token lengths چھوٹے □Model کم □ہوتا جاتا □□□ relative کے patterns □زیادہ کو follow □□ کرتا، drift کو compound □□□ کرتا outputs

---

سوال 76

صورتحال: Users □مبہم requests کرتے □ہیں جیسے □"کیا آپ report □ہیں؟ کر سکتے □میں مدد Assistant کئی جس سے 40 %، abandonment □□□ ہوتا□ سوالات پوچھ کر respond □□ کرتا (کون سی report؟ کیا مدد ؟ deadline □□؟ (کیا

□بہترین solution کیا □□؟

- A) Reasonable assumptions □انہیں، کریں explicitly اور بیان کریں، adjustments **[درست]** □کی پیشکش کریں
- B) Respond □پہلے سے کرنے smaller model ساتھ کے ambiguity classify □کریں
- C) Assumptions بغیر □کیے بیان predefined interpretations □استعمال کریں
- D) Assistant □کو turn ایک میں clarifying question □تک محدود کریں

A کیوں: Reasonable stated assumptions بڑھنا آگے□ ساتھ □کہ user کو informed اور in control □ہوئے □رکھتے□ back-and-forth □□□ کو مکمل طور پر ختم کرتا Silent predefined interpretations (C) users کو confuse کرتی □یں جب response □سے □ارادے □کہ ان match □ہو □نہ One-question limit (D) پھر بھی back-and-forth turns کی □□□ ضرورت Smaller classification model (B) core UX بغیر □حل کیے مسئلے latency اور infrastructure complexity □□□ شامل کرتا

عملی مشقیں

Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration، structured errors، اور escalation ساتھ □کہ agent loop □بنائیں

Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md، custom commands، path-specific rules، اور MCP servers configure □کریں

Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema، validation/retry loops، اور Message Batches API ساتھ □کہ extraction pipeline □بنائیں

Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration، explicit context passing، error propagation، اور source tracking implement □کریں

Appendix: Technologies and Concepts

Technology	Key aspects
Claude Agent SDK	agent loops, `stop_reason`, hooks, `Task`, `allowedTools`
MCP	servers, tools, resources, `isError`, `.mcp.json`
Claude Code	CLAUDE.md, `.claude/rules/`, `.claude/commands/`, `.claude/skills/`, planning mode
Claude Code CLI	`-p`, `--output-format json`, `--json-schema`
Claude API	`tool_use`, `tool_choice`, `stop_reason`, `max_tokens`

Message Batches API	50% savings, 24-hour window, `custom_id`
JSON Schema	required/optional, nullable, enum values
Few-shot prompting	ambiguous scenarios □ لیے □ کے examples
Prompt chaining	sequential decomposition
Context window	token budget, lost-in-the-middle
Confidence calibration	field-level scoring, stratified sampling

---

Out-of-Scope Topics

یہ □ موضوعات امتحان میں شامل نہ□یں □یں:

- Claude models کی fine-tuning
- Authentication، billing، یا account management
- مخصوص programming languages کی تفصیلی implementation
- MCP servers کی hosting/infrastructure
- Claude کی internal architecture یا model weights
- Constitutional AI، RLHF، یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

Preparation Recommendations

1. Claude Agent SDK ایک ساتھ □ کے agent بنائیں
2. Claude Code کو real project پر configure کریں
3. MCP tools design اور test کریں
4. Data extraction pipeline بنائیں
5. Prompt engineering practice کریں
6. Context management patterns سیکھیں

7. Escalation اور human-in-the-loop workflows سمجھیں
8. Practice exam حل کریں